arXiv:2307.14840v5 [quant-ph] 4 Dec 2024

State preparation by shallow circuits using feed forward

Harry Buhrman[1], Marten Folkertsma[1], Bruno Loff[2], and Niels M. P. Neumann[1,3]

[1]QuSoft, CWI & University of Amsterdam, Amsterdam, the Netherlands
[2]LASIGE & Department of Mathematics, University of Lisbon
[3]The Netherlands Organisation for Applied Scientific Research (TNO), Delft, the Netherlands

Fault tolerant quantum computers repetitively apply a four-step procedure: First, perform a few one and two-qubit quantum gates. Second, perform a syndrome measurement on a subset of the qubits. Third, perform fast classical computations to establish if and where errors occurred. And, fourth, correct the errors with a correction step. The next iteration applies the same procedure with new one and two-qubit gates. Even though current error-rates prohibit this procedure to work and fault tolerant quantum computing remains a distant goal, the same procedure can already prove useful today. In this work we make use of this four-step scheme not to carry out fault-tolerant computations, but to enhance short, *constant*-depth, quantum circuits that perform 1 qubit gates and *nearest-neighbor* 2 qubit gates.

We introduce a new computational model called *Local Alternating Quantum Classical Computations* (LAQCC). In this model, qubits are placed in a grid and they can only interact with their direct neighbors; the quantum circuits are of constant depth with intermediate measurements; a classical controller can perform log-depth computations on these intermediate measurement outcomes and control future quantum operations based on the outcome. This model fits naturally between quantum algorithms in the NISQ era and full-fledged fault-tolerant quantum computation. We first prove that any Clifford circuit has an equivalent LAQCC circuit, and that any LAQCC circuit can be simulated by a QNC^1 circuit. Next, we conjecture the non-simulatability of LAQCC by showing that LAQCC contains the class of Instantaneous Quantum Polynomial-time circuits. We also show that any LAQCC circuit with polynomial-sized quantum circuits and unbounded classical computations is contained in the class of quantum circuits equipped with post-selection gates with respect to the task of state preparation. We continue by presenting LAQCC implementations of different subroutines, including OR-gates, quantum Fourier transforms and Threshold gates. These subroutines prove vital in constructing three state preparation routines in the main part of this work. Preparing a uniform superposition uses constant-depth arithmetic gates, combined with an exact Grover implementation by Long. For the W-state, we employ a compress-uncompress method to switch between a binary and one-hot encoding. This method extends to the more generalized Dicke-states, the superposition of n-bit strings of Hamming weight k, for $k = \mathcal{O}(\sqrt{n})$, but fails for higher k due to the birthday paradox. We extend this protocol to a protocol that prepares many-body scar states, highly excited states with low entanglement and longer coherence times than states with the same energy density. We present a circuit for preparing Dicke-states for larger k requiring log-depth circuits that maps between the factoradic number system and the combinatorial number system.

1 Introduction

Current quantum hardware is unable to carry out universal quantum computations due to the buildup of errors that occur during the computation. The magnitude of the individual error is currently above the value that the Threshold Theorem requires in order to kick-start quantum

error correction and fault-tolerant quantum computation [NC10, Section 10.6]. Although the experimentally achieved fidelity rates are promising and the error bounds are inching closer to the required threshold, we will have to work for the foreseeable future with quantum hardware with errors that build-up during the computation. This implies that we can only do a limited number of steps before the output of the computation has become completely uncorrelated with the intended one.

For fault-tolerant quantum computing, we repeat four steps: 1) We apply a number of single and two-qubit quantum gates, in parallel whenever possible; 2) We perform a syndrome measurement on a subset of the qubits; 3) We perform fast classical computations to determine which errors have occurred and how to correct them; and, 4) We apply correction terms based on the classical computations. We then repeat these four steps with a next sequence of gates. These four steps are essential to fault-tolerant quantum computing.

The starting point of this work is to use the four steps outlined above, not to carry out error correction and fault-tolerant computation, but to enhance short, constant-depth, *uncorrected* quantum circuits that perform single qubit gates and *nearest-neighbor* two qubit gates. Since in the long run we will have to implement error-correction and fault-tolerant computation anyhow, and this is done by such a four-step process, why not make other use of this architecture? Moreover, on some of the quantum hardware platforms, these operations are already in place. Embracing this idea we naturally arrive at the question: what is the computational power of *low-depth* quantum-classical circuits organized as in the four steps outlined above? We thus investigate circuits that execute a small, ideally constant, number of stages, where at each stage we may apply, in parallel, single qubit gates and *nearest-neighbor* two qubit gates, followed by measurements, followed by low-depth classical computations of which the outcome can control quantum gates in later stages. It is not clear, at first, whether such circuits, especially with constant depth, can do anything remotely useful. But we will see that this is indeed the case: many quantum computations can be done by such circuits in constant depth. By parallelizing quantum computations in this way, we improve the overall computational capabilities of these circuits, as we do not incur errors on qubits that are idle, simply because qubits are not idle for a very long time. Furthermore, reducing the depth of quantum circuits, at the cost of increasing width, allows the circuit to be run faster even if errors occur.

The first usage of such a four-step layout, not to do error correction, but to perform computations, can be found in the paradigm of measurement-based quantum computing [GC99; RB01; Joz06; CJL08]: A universal form of quantum computing where a quantum state is prepared and operations are performed by measuring qubits in different bases, depending on previous measurements and intermediate measurements.

Pham and Svore were the first to formalize the four-step protocol for performing computations [PS13]. They included specific hardware topologies by considering two-dimensional graphs for imposing constraints on qubit interactions. In their model, they develop circuits for particularly useful multi-qubit gates, including specifying costs in the width, number of qubits, depth, number of concurrent time steps, size, and total number of non-Identity operations. As a result, they find an algorithm that factors integers in polylogarithmic depth. Browne, Kashefi, and Perdrix showed that the main tool in the work by Pham and Svore, the fan-out gate, can also be replaced by additional log-depth classical computations in the measurement-based quantum computing setting [BKP11].

More recently, Piroli, Styliaris, and Cirac introduced a scheme to implement unitary operations involving quantum circuits combined with Local Operations and Classical Communication (LOCC) channels: LOCC-assisted quantum circuits [PSC21]. Similarly to the four-step scheme we just described, they allow for a short depth circuit to be run on the qubits, followed by one round of LOCC, in which ancilla qubits are measured and local unitaries are applied based on the measurement outcomes. They show that in this model any 1D transitionally invariant matrix-product state (MPS) with fixed bond dimension is in the same phase of matter as the trivial state. Similar ideas can be found in [TVV23a; Tan+24].

In this work, we introduce a new model, called *Local Alternating Quantum-Classical Computations* (LAQCC). In this model we alternate between running quantum circuits (constrained by locality), ending in the measurement of a subset of qubits, and fast classical computations based on the measurement results. The outcome of the classical computations are then used to control

future quantum circuits. We allow for flexibility in this model, by giving different constraints to the power of both the quantum circuits and the classical circuits as well as the number of alternations between them. Most attention will be given to LAQCC containing quantum circuits of constant depth, classical circuits of logarithmic depth and at most a constant number of alternations between them. Any circuit constructed in this model is considered to be of constant depth. We restrict ourselves to logarithmic depth classical computations, as this is the first natural and non-trivial extension beyond constant-depth classical computations. Constant-depth classical computations do however also have an equivalent constant-depth quantum implementation.

The definition of LAQCC sharpens the original definition of Pham and Svore by adding constraints to the intermediate classical computations. This allows us to bound the power of LAQCC from above.

The main result of Piroli, Styliaris, and Cirac, that 1D translational invariant MPS with fixed bond dimension can be prepared by LOCC-assisted circuits, relies on local symmetries of the MPS. These symmetries allow them to prepare local states (on a constant number of qubits) and glue them together by doing one round of the appropriate entangling measurement and corrections, after which they run a round of local unitaries to get the desired result. This general scheme for preparing states that exhibit an MPS description with the appropriate local symmetries requires only geometrically local unitaries and one round of measurement and corrections an therefore is accessible in LAQCC. Studying different local symmetries, known as Symmetry Protected Topological (SPT) phases of matter, to find measurement-based constant depth circuits for states is a broad ongoing field of research [TVV23a; Tan+24; Smi+23]. All these schemes have a LAQCC implementation.

Note however that Piroli, Styliaris, and Cirac also suggest a circuit for the W-state. This circuit uses sequentially and dependent measurement-based corrections of the ancilla qubits. These dependent measurements translate to sequential alternations between the quantum and classical circuits and therefore increase the total depth to linear depth, exceeding the constant-depth constraints imposed by LAQCC-circuits.

We study the power of the LAQCC model with respect to state preparation, showing that even with only constant quantum-depth and logarithmic classical depth it remains possible to prepare states with long-range entanglement. Another surprising result is that it is unlikely that LAQCC circuits are classically simulatable. We show that any instantaneous quantum polynomial-time (IQP) circuit [BJS10; SB09] has an LAQCC implementation. Classical simulation of IQP circuits implies the collapse of the polynomial hierarchy to the third level, which is not believed to be true [BMS17]. Therefore, we expect that LAQCC circuits are unlikely to be classically simulatable. We bound the power of LAQCC by showing that it is contained in QNC^1, the class of polynomial-size, log-depth circuits.

Next, we also study the power that intermediate classical calculations can add to quantum computations, by considering a new model that alternates between polynomially many polynomial-depth quantum circuits and unbounded classical computations We study this model by doing a complexity theoretical analysis, where we draw inspiration from the notions of complexity given by Rosenthal and Yuen, Metger and Yuen, and Aaronson. All three complexity notions are based on the notion of state preparation, instead of more traditional definition of complexity such as the decidability of a computational problem. The first two consider classes based on sequences of quantum states preparable by a polynomial-sized quantum circuit, where the circuits are uniformly generated by a computational class, for instance, the class PSPACE, which results in the complexity class StatePSPACE [RY22; MY23]. The third notion considers a relative complexity, where the complexity is measured between two given states, and is measured by the number of gates, from a given gate-set, required to transform one state in another state [Aar04]. For our definition of state preparation complexity, we drop the uniformity constraint from [RY22; MY23] and define a class as StateX, which refers to states preparable by circuits of type X. As an example, if $\mathsf{X} = \mathsf{QNC}^0$, this results in the class $\mathsf{StateQNC}^0$, which is the set of states preparable from the $|0\rangle^n$ state by poly-size constant-depth circuits. This notion is similar to the relative complexity from [Aar04], where one state is the $|0\rangle^n$ state and instead of counting the number of gates we consider the set of states preparable by a fixed number of gates. Using this notion of complexity we show that any state preparable by an LAQCC^* circuit is also preparable by a PostQPoly circuit, the class of circuits of polynomial depth with an additional post-selection gate.

All Clifford circuits have a constant-depth LAQCC implementation, implying that any stabilizer state can be implemented by a constant-depth LAQCC circuit, see Section 3.2 for a proof of this statement. Efficient circuits for stabilizer states have been known already through measurement-based quantum computing. Therefore this paper focuses on the preparation of non-stabilizer states, and as a surprising result we find novel constant-depth protocols for four very natural classes of non-stabilizer states. Despite the extensive research into these four classes of non-stabilizer states and the many applications of them, no efficient constant- or low-depth state preparation protocols are known yet. We specifically consider these four classes as they are all often used as initial states in other algorithms.

The first state is a uniform superposition over an arbitrary number of states. This state finds applications in many quantum algorithms, as they often start with a uniform superposition over multiple states. This superposition is often achieved by applying Hadamard gates to every qubit due to its simplicity to prepare. Yet, the analysis of many algorithms, such as Shor's algorithm [Sho97], would benefit from a different initial superposition. The circuit to prepare the uniform superposition over an arbitrary number of states uses an exact version of Grover search as a subroutine, that turns a probabilistic circuit, with a known constant probability of success, into a deterministic circuit. We use the circuit for preparing a uniform superposition over an arbitrary number of states as a subroutine in the next two quantum state preparation protocols.

The second state is the W-state, the uniform superposition over all computational basis states of Hamming-weight 1, a natural long-ranged entangled state that displays a fundamentally nonequivalent type of entanglement from the Greenberger–Horne–Zeilinger state [DVC00], for which LAQCC-type constant-depth circuits were previously known [PS13; PSC21]. The W-state is often used as benchmark for new quantum hardware [Häf+05; Nee+10; Gar+23]. A novel way to prepare the W-state therefore gives a new way to benchmark different quantum devices with each other. A circuit for preparing the W-state was given in [PSC21], but this implementation requires sequentially alternating measurements followed by local unitaries, which in the LAQCC model is not considered to be of constant depth. We improve this protocol by giving an LAQCC implementation of the W-state, based on a compress-uncompress method that links the one-hot and binary encoding of integers.

The third state considered is the Dicke state, a generalization of the W-state, a superposition over all computational basis states with Hamming-weight k [Dic54]. Dicke states have relevance in various practical settings. For instance, for quantum game theory [ÖSI07], quantum storage [BCH06; PB10], quantum error correction [Ouy14], quantum metrology [Tót12], and quantum networking [Pre+09]. Dicke states have been used as a starting state for variational optimization algorithms, most notably Quantum Alternating Operator Ansatz (QAOA) [Had+19], to find solutions to problems such as Maximum k-vertex Cover [Bra+22; CEB20]. The ground states of physical Hamiltonians describing one-dimensional chains tend to show a resemblance to Dicke states such as states resulting from the Bethe ansatz, making them an ideal starting state when investigating the ground state behavior of these Hamiltonians [TDL10; Bra13; Bez+21]. For instance, the algorithm by Van Dyke et al., who give an algorithm to prepare the Bethe ansatz eigenstates of the spin-1/2 XXZ spin chain, starts by first preparing a Dicke state [Van+21]. A Dicke-state preparation protocol based on the compress-uncompress methodology used in the W-state furthermore finds applications in entanglement distillation, where the entanglement of a large state is concentrated on only a few qubits. Efficient deterministic circuits for preparing Dicke states have been proposed by Bärtschi and Eidenbenz [BE19; BE22]. They provide a quantum circuit of depth $\mathcal{O}(k \log(\frac{n}{k}))$, allowing arbitrary connectivity, to prepare a Dicke state, which they conjecture to be optimal when k is constant. In this work, we provide a constant-depth LAQCC circuit below their conjectured bound already for constant k. However, this does not directly disprove their conjecture, as we allow for intermediate measurements and classical computations. More significantly, we even construct constant-depth LAQCC circuits for $k = \mathcal{O}(\sqrt{n})$ greatly improving their bound. This construction extends the compress-uncompress method for the W-state combined with additional subroutines.

We continue with a log-depth state preparation protocol for the Dicke-state for arbitrary k. This protocol implements an efficient transformation between the factoradic number representation and the combinatorial number representation of a positive integer. The combinatorial number representation relates directly to the Dicke state. The provided efficient transformation between

number representation systems might be of independent interest.

We conclude by modifying our protocol for preparing a Dicke-state to a protocol that prepares quantum many-body scar states in constant-depth. These states have low entanglement and longer coherence times than states with similar energy density. These characteristics make many-body scar states interesting to analyze and relevant within physics. Many-body scar states appear for instance in the AKLT model [Aff+87; Mou+18; MRB18] and different spin models [SI19; Mou+20]. Known methods for preparing these states have polynomial-depth [Gus+23], whereas our circuit has constant depth.

Summary of results

- We give a new definition of a computational model that captures the power of the four step process: applying a constant number of layers of one- and two-qubit gates; performing a syndrome measurement; perform a fast classical computation determining corrections; apply corrections. We call this model *Local Alternating Quantum Classical Computations*, or LAQCC for short. In this model we bound the allowed quantum operations, intermediate classical calculations, and number of rounds separately. In Section 3 we define this model and give a list of operations based on results from literature contained in this computational model. In some of these operations we explicitly use that we allow for multiple, but at most constant, rounds of corrections.
- We show show that there exist LAQCC circuits that can not be weakly simulated in Section 3.4. We further show that for every LAQCC circuit there exists a QNC^1 circuit simulating it perfectly, in Section 3.5.
- We introduce a new type computational complexity for preparing states and show that the extension of LAQCC where we allow a polynomial number of rounds and unbounded classical computation, is contained in PostQPoly, the class of polynomial circuits with post-selection, in Section 3.6.
- We show a protocol to prepare the uniform superposition state of size q in LAQCC using $\mathcal{O}(\lceil \log_2(q) \rceil^2)$ qubits in Section 4.1.
- We show a protocol to prepare the W_n state in LAQCC using $\mathcal{O}(n \log(n))$ qubits in Section 4.2.
- We show two ways of preparing the Dicke-(n,k) state. The first method is in LAQCC, works up to $k = \mathcal{O}(\sqrt{n})$, uses $\mathcal{O}(n^2 \log(n))$ qubits, and is found in Section 4.3. The second method is in LAQCC-LOG (an extension of LAQCC allowing for logarithmic number of alterations instead of constant), works for any k, uses $\mathcal{O}(\text{poly}(n))$ qubits, and is found in Section 4.4.
- We extend on our LAQCC method of generating Dicke-(n,k) states for $k = \mathcal{O}(\sqrt{n})$ and show a protocol to generate many-body scar states for a particular Hamiltonian in LAQCC (Section 4.5).

Summarized in a table, we provide the following state generation protocols: In the entry for the

State description	Width	Depth	Implementation
Uniform superposition mod q: $\frac{1}{\sqrt{q}} \sum_{i=0}^{q-1} \lvert i \rangle$	$\mathcal{O}(\lceil \log^2 q \rceil)$	$\mathcal{O}(1)$	Section 4.1
W-state: $\frac{1}{\sqrt{n}} \sum_{i=0}^{n-1} \lvert e_i \rangle$	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	Section 4.2
Dicke-(n,k), $k = \mathcal{O}(\sqrt{n})$: $\binom{n}{k}^{-1/2} \sum_{x \in \{0,1\}^n : \lvert x \rvert = k} \lvert x \rangle$	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(1)$	Section 4.3
Dicke-(n,k): $\binom{n}{k}^{-1/2} \sum_{x \in \{0,1\}^n : \lvert x \rvert = k} \lvert x \rangle$	$\mathcal{O}(\text{poly}(n))$	$\mathcal{O}(\log n)$	Section 4.4
QMBS: $\lvert S_k \rangle = \frac{1}{k! \sqrt{\mathcal{N}(n,k)}} (Q^\dagger)^k \lvert \Omega \rangle$	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(1)$	Section 4.5

Table 1: Summary of state preparation protocols given in this paper.

quantum many-body scar state Q denotes the raising operator and $\mathcal{N}(n,k) = \binom{n-k-1}{k}$. Section 4.5 will provide more details on the variables and the implementation.

Organization of the paper We first introduce relevant preliminaries in Section 2. In Section 3 we formally define the class of Local Alternating Quantum-Classical Computations (LAQCC). We also show that any Clifford circuit can be implemented in constant depth LAQCC (a result based on a result from measurement-based quantum computing [Joz06]). This result allows us to give many useful multi-qubit gates and routines in Section 3.3. Beyond that we show that constant depth LAQCC circuits are contained in QNC^1 and that any IQP circuit has an LAQCC implementation. We conclude this section with an analysis of a more powerful instantiation of LAQCC and show an inclusion with respect to the class PostQPoly, which is the class of circuits of polynomial depth with one additional post-selection gate. In Section 4 we give LAQCC circuit implementations for preparing the uniform superposition over an arbitrary number of states, the W-state and the Dicke state up to $k = \mathcal{O}(\sqrt{n})$. We furthermore give a log-depth circuit implementation for preparing the Dicke state for any k. We conclude by showing a LAQCC circuit for generating many body scar states of a particular type of Hamiltonian.

2 Preliminaries

In this section we recap definitions used throughout the rest of the paper.

2.1 Complexity classes

The computational model introduced in this work uses complexity classes. Typically these classes are defined as classes of decision problems solvable by some type of circuits. Below we give definitions of some of these complexity classes in terms of the circuits contained in that class.

Definition 2.1. The class NC^k consists of all decision problems solvable by circuits of $\mathcal{O}((\log n)^k)$ depth and polynomial size, and consisting of bounded-fan-in AND- and OR-gates.

The class AC^k consists of all decision problems solvable by circuits of $\mathcal{O}((\log n)^k)$ depth and polynomial size, and consisting of unbounded-fan-in AND- and OR-gates.

The class TC^k consists of all decision problems solvable by circuits of $\mathcal{O}((\log n)^k)$ depth and polynomial size, and consisting of unbounded-fan-in AND-, OR- and Threshold$_t$-gates, where a Threshold$_t$-gate evaluates to one if and only if the sum of the inputs is at least t.

These classes also have a quantum equivalent class.

Definition 2.2. The class QNC^k consists of all decision problems solvable by quantum circuits of $\mathcal{O}((\log n)^k)$ depth and polynomial size, and consisting of single- and two-qubit quantum gates.

Definitions for the quantum versions of AC^k and TC^k also exist. However, when equipping the class QNC^k with unbounded-fan-in parity gates, all three classes intersect [Gre+02; Moo99; TT13].

Two other often used classes are P and L.

Definition 2.3. The class P consists of all decision problems solvable in polynomial time by a Turing machine.

The class L consists of all decision problems solvable using only a logarithmic amount of memory.

The first class poses a limit on the depth of the operations performed by the Turing machine. The second class instead limits the available memory. Note however that with a logarithmic amount of memory, only a polynomial number of states is available, and hence the computation time is polynomial as well.

Relations between different complexity classes exists: for instance, for all k, $\mathsf{NC}^k \subseteq \mathsf{AC}^k \subseteq \mathsf{TC}^k$. By Johnson, we also have the inclusion $\mathsf{L} \subseteq \mathsf{AC}^1 \subseteq \mathsf{TC}^1$ [Joh90].

In the remainder of this work we abuse notation and refer to X-circuits as circuits that correspond to a decision problem in the class X. For example, an NC^k circuit is a circuit of $\mathcal{O}((\log n)^k)$ depth and polynomial size that corresponds to a decision problem in NC^k.

Finally, we define the class of polynomial sized quantum circuits: [1]

Definition 2.4. The class $\mathsf{QPoly}(n)$ consists of all polynomial-sized quantum circuits (in n) that use single and two-qubit quantum gates.

[1] Due to the bounded-error-aspect associated to decision problems in BQP, we follow this definition instead of talking about BQP-circuits.

2.2 Quantum gate sets

First, recall the definition of the Pauli group and the Clifford group.

Definition 2.5. The one qubit Pauli-group P_1 consists of four matrices, the identity matrix I and the three Pauli matrices:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

together with a global phase of ± 1 or $\pm i$.

The n-qubit Pauli-group P_n is the set of all 4^{n+1} possible tensors of length n of matrices from P_1, together with a global phase of ± 1 or $\pm i$.

The Clifford group forms the other well-known group of quantum circuits, as it stabilizes the Pauli group.

Definition 2.6. The Clifford group C_n consists of all n-qubit unitaries that leave the Pauli group P_n invariant under conjugation. That is, let $c \in C_n$ be any Clifford circuit, then for any $P \in P_n$, there exists a $P' \in P_n$, such that $cP = P'c$.

The Clifford group is generated by the $CNOT$-gate, the Hadamard gate H and the phase gate S, that act on computational basis states $|x\rangle$ and $|y\rangle$ via

$$CNOT : |x\rangle |y\rangle \mapsto |x\rangle |y \oplus x\rangle , \qquad H : |x\rangle \mapsto |0\rangle + (-1)^x |1\rangle , \qquad S : |x\rangle \mapsto i^x |x\rangle .$$

Any circuit constructed using only these three gates is called a Clifford circuit.

Unsurprisingly, Clifford circuits only cover a small part of the possible quantum circuits. Moreover, on a linear nearest-neighbor architecture, $\mathcal{O}(n)$ deep Clifford circuits suffice to simulate any Clifford unitary of size $2^n \times 2^n$ [MR18]. We can furthermore simulate Clifford circuits efficiently [Got98]. Universal quantum computations require additional gates, though almost any quantum gate suffices. For example, adding the single qubit T-gate, $T : |x\rangle \mapsto e^{i\pi x/4} |x\rangle$, to the Clifford group gives a universal gate set.

2.3 Two quantum subroutines

This section discusses two quantum subroutines used in later sections. The first concerns Grover's algorithm with zero failure probability [Lon01]. The second concerns parallelization of commuting gates using quantum fan-out gates [HŠ05].

Grover's search algorithm gives a quadratic speed-up for unstructured search [Gro96]. After sufficient iterations, a measurement returns a target state with high probability. Surprisingly, if the exact number of target states is known, a slight modification of the Grover iterates allows for returning a target state with certainty, assuming noiseless computations. Lemma 3.14 uses the next lemma to prepare quantum states instead of to find a target state.

Lemma 2.7 ([Lon01])**.** *Let L be a set of items and $T \subseteq L$ a set of targets, with $N = |L|$ and $m = |T|$ both known. Let $g : L \to \{0,1\}$ label the items in L and define the oracle $O_g : |x\rangle |b\rangle \mapsto |x\rangle |b \oplus g(x)\rangle$.*

Then, there exists a quantum amplitude amplification algorithm that makes $\mathcal{O}(\sqrt{N/m})$ queries to O_g and prepares the quantum state $\frac{1}{\sqrt{|T|}} \sum_{x \in T} |x\rangle$.

For the other result, we use the quantum fan-out gate, that implements the map

$$|x\rangle |y_1\rangle \dots |y_n\rangle \mapsto |x\rangle |y_1 \oplus x\rangle \dots |y_n \oplus x\rangle .$$

Section 3.3 gives more details on how to implement this gate. Høyer and Špalek introduced this gate and analyzed its properties. The state preparation protocols given in Section 4 use the property that the quantum fan-out gate allows parallelization of commuting quantum gates [HŠ05].

Lemma 2.8. ([HŠ05, Theorem 3.2]) *Let $\{U_i\}_{i=1}^n$ be a pairwise commuting set of gates on k qubits. Let $U_i^{x_i}$ be the gate U_i controlled by qubit $|x_i\rangle$. Let T be the unitary that mutually diagonalizes all U_i. Then there exists a quantum circuit, using quantum fan-out gates, computing $U = \prod_{i=1}^n U_i^{x_i}$ with depth $\max_{i=1}^n \operatorname{depth}(U_i) + 4 \cdot \operatorname{depth}(T) + 2$ and size $\sum_{i=1}^n \operatorname{size}(U_i) + (2n+2) \cdot \operatorname{size}(T) + 2n$, using $(n-1)k$ ancilla qubits.*

3 The LAQCC model

This section formally defines the LAQCC model. Next, we show that all Clifford circuits have an efficient and equivalent LAQCC circuit. We then give quantum gates and tools constructable within LAQCC, such as the quantum fan-out gate and weighted threshold gate, and we conclude by showing that any LAQCC-circuit has an equivalent QNC^1-circuit.

3.1 Model definition

We define the computational model *Local Alternating Quantum-Classical Computations* (LAQCC) as follows:

Definition 3.1 (Local Alternating Quantum-Classical computations)**.** Let $\mathsf{LAQCC}(\mathcal{Q}, \mathcal{C}, d)$ be the class of circuits such that

- every quantum layer implements a quantum circuit $Q \in \mathcal{Q}$ constrained to a grid topology;
- every classical layer implements a classical circuit $C \in \mathcal{C}$;
- there are d alternating layers of quantum and classical circuits;
- after every quantum circuit Q a subset of the qubits is measured;
- the classical circuit receives input from the measurement outcomes of previous quantum layers;
- the classical circuit can control quantum operations in future layers.

The allowed gates in the quantum and classical layers are given by $\mathcal{Q}$ and $\mathcal{C}$ respectively. Furthermore, we require a circuit in $\mathsf{LAQCC}(\mathcal{Q}, \mathcal{C}, d)$ to deterministically prepare a pure state on the all-zeroes initial state.

The grid topology imposed on the quantum operations implies that qubits can only interact with their direct neighbors on the grid. A circuit in $\mathsf{LAQCC}(\mathcal{Q}, \mathcal{C}, d)$ can use the results of the classical intermediate layers and control quantum operations in future layers. In a sense, information is fed forward in the circuit. Note, as classical computations are in general significantly faster than the quantum operations, we only count the quantum operations towards the depth of the circuit, unless specified otherwise.

Remark 3.2. *Note that there exists ambiguity in the choices for $\mathcal{Q}$ and $\mathcal{C}$. For example, we have $\mathsf{LAQCC}(\mathsf{QPoly}(n), \mathsf{P}, \mathcal{O}(1)) = \mathsf{LAQCC}(\mathsf{QNC}^0, \mathsf{P}, \mathcal{O}(\mathrm{poly}(n)))$. This follows as any P-circuit is in $\mathsf{QPoly}(n)$, and we can concatenate $\mathrm{poly}(n)$ constant-depth quantum circuits with trivial intermediate classical computations.*

This ambiguity is non-trivial: consider for instance

$$\mathsf{LAQCC}(\mathsf{QNC}^1, \mathsf{NC}^1, \mathcal{O}(1)) \subseteq \mathsf{LAQCC}(\mathsf{QNC}^0, \mathsf{NC}^1, \mathcal{O}(\log(n))).$$

The inclusion from left to right follows immediately by same argument as above. It is however not obvious if the logarithmic number of measurement rounds, allowed in the right hand side, can be simulated by a QNC^1 circuit. Even stronger, we will show in Section 3.3 that threshold gates are available in $\mathsf{LAQCC}(\mathsf{QNC}^0, \mathsf{NC}^1, \mathcal{O}(1))$. From this fact it follows immediately that any TC^1-circuit is contained in $\mathsf{LAQCC}(\mathsf{QNC}^0, \mathsf{NC}^1, \mathcal{O}(\log(n)))$. It is unclear these circuits are also contained in $\mathsf{LAQCC}(\mathsf{QNC}^1, \mathsf{NC}^1, \mathcal{O}(1))$.

In the remainder of this work, we consider a specific instantiation of $\mathsf{LAQCC}(\mathcal{Q}, \mathcal{C}, d)$.

Notation 3.3. We let LAQCC refer to the instance $\mathsf{LAQCC}(\mathsf{QNC}^0, \mathsf{NC}^1, \mathcal{O}(1))$, together with a grid nearest-neighbor topology and a quantum gate-set of all single-qubit gates and the two-qubit CNOT gate. The classical computations are bounded to logarithmic depth and of bounded-fan-in.

The class NC^1 is a natural non-trivial class beyond constant-depth complexity classes. As the depth of these circuits remains low, they can be implemented quickly. In this work, we assume qubits will not decohere during the classical computation. Incorporating the errors throughout the whole LAQCC-computation, including the classical intermediate computations, proves an interesting direction for future research. Interesting, LAQCC contains many useful gates and subroutines already, as outlined in the next section, specifically the fan-out gate, implying directly that threshold gates are also contained in LAQCC.

In its current definition, $\mathsf{LAQCC}(\mathcal{Q}, \mathcal{C}, d)$, and hence also LAQCC, are classes of circuits. When considering the capabilities of $\mathsf{LAQCC}(\mathcal{Q}, \mathcal{C}, d)$ in preparing states, it is helpful to define a related class that consists of states preparable by a circuit in $\mathsf{LAQCC}(\mathcal{Q}, \mathcal{C}, d)$.

Definition 3.4. Let $\mathcal{H}_n$ be a Hilbert space on n qubits, then define

$$\mathsf{StateX}_{n,\varepsilon} = \{|\psi\rangle \in \mathcal{H}_n \mid \exists\, \mathsf{X}\text{-circuit } A : \langle\psi| A |0\rangle^{\otimes n} \geq 1 - \varepsilon\}.$$

This is the subset of n-qubit states $|\psi\rangle$ such that there exists a circuit corresponding to the class X that prepares a quantum state that has inner product at least $1 - \varepsilon$ with $|\psi\rangle$.

Define $\mathsf{StateX}_\varepsilon = \bigcup_{n\in\mathbb{N}} \mathsf{StateX}_{n,\varepsilon}$.

This definition extends already existing ideas and definitions of state-complexity [AAS20; RY22; Sus18]. Our definition is very similar to state complexity defined in [MY23], where we are interested in which states are contained in a class, however we drop the uniformity requirement and instead study the set of states that can be generated by a specific class of circuits. An example of a circuit class is $\mathsf{StateLAQCC}(\mathcal{Q}, \mathcal{C}, d)_{n,\varepsilon}$.

Notation 3.5. The class $\mathsf{StateLAQCC}(\mathcal{Q}, \mathcal{C}, d)_{n,\varepsilon}$ consists of all n-qubit states $|\psi\rangle$ for which an $\mathsf{LAQCC}(\mathcal{Q}, \mathcal{C}, d)$ exists that prepares a state that has inner product at least $1 - \varepsilon$ with $|\psi\rangle$.

Another example is the circuit class of PostQPoly.

Definition 3.6. The class PostQPoly consists of all polynomial-sized quantum circuits with one extra qubit, where the outcome state is considered conditional on the extra qubit being in the one state. If the extra qubit is in the zero state, the output state may be anything.

The class $\mathsf{StatePostQPoly}_{n,\varepsilon}$ consists of all n-qubit states $|\psi\rangle$ for which a polynomial-sized quantum circuit exists that prepares a state that, conditional on the extra qubit being one, has inner product at least $1 - \varepsilon$ with $|\psi\rangle$.

The next section shows that all polynomial-size Clifford circuits are in LAQCC. As a result, the quantum fan-out gate is also in LAQCC, a multi-qubit gate which enables many different subroutines (by applying Lemma 2.8). We give a list of such quantum gates and quantum subroutines accessible LAQCC in Section 3.3. We conclude by showing that any LAQCC-circuit corresponds to a QNC^1-circuit.

3.2 Clifford circuits

The concept of intermediate measurements with subsequent computations is closely related to measurement-based quantum computing. A famous result from measurement-based quantum computing us that all Clifford circuits can be paralellized using measurements. In this section we borrow techniques from this result to show that any Clifford circuit has an LAQCC implementation.

This result is best understood in the teleportation based quantum computing model [Joz06], a specific instance of measurement-based quantum computing that applies quantum operations using bell measurements. In teleportation, qubits are measured in the Bell basis, which projects the measured qubits onto an entangled two-qubit, or ebit, state, up to local Pauli gates. This projection combined with an ebit state teleports a quantum state between qubits. After teleportation, one needs to correct the local Pauli gate created by the bell measurement. A similar process can be used to apply quantum gates. However, the Pauli gates that arise during teleportation have to be corrected before the calculations can proceed, which necessitates subsequent adaptive operations.

With Clifford circuits, these subsequent operations can be omitted. Clifford circuits stabilize the Pauli group, which allows for simultaneous measurements and hence parallelization of the full

Clifford circuit [Joz06]. Consider a simple example of teleporting a single-qubit quantum state. A Bell basis measurement projects two qubits on $\sum_{i\in\{0,1\}} \langle ii| P^{a,b} \otimes I$, where $P^{a,b} = Z^a X^b$ and $a, b \in \{0, 1\}$ correspond to the four possible measurement outcomes.

By using one Bell-basis measurement, we can apply two sequential Clifford gates U_1 and U_2 on a quantum state $|\psi\rangle$, which gives:

$$\sum_{i,j\in\{0,1\}} \left[(\langle ii| (P^{a,b} \otimes I) \otimes I\right] U_1 \otimes I \otimes U_2 |\psi\rangle |jj\rangle = \sum_{i,j\in\{0,1\}} \langle i| P^{a,b} U_1 |\psi\rangle \langle i|j\rangle U_2 |j\rangle$$
$$= \sum_{i\in\{0,1\}} U_2 |i\rangle \langle i| P^{a,b} U_1 |\psi\rangle = U_2 P^{a,b} U_1 |\psi\rangle .$$

Note that besides projecting on a Bell state, an initial entangled Bell-state is required. U_2 is a Clifford gate, hence there exists a $P^{\hat{a},\hat{b}}$ such that $U_2 P^{a,b} U_1 |\psi\rangle = P^{\hat{a},\hat{b}} U_2 U_1 |\psi\rangle$, allowing the correction term to be pushed to the end of the circuit. Repeating the same argument for multiple Clifford unitaries gives the quantum state $...P_2^{a_2,b_2} U_2 P_1^{a_1,b_1} U_1 |\psi\rangle$. Due to the conjugation relation of Clifford and Pauli gates, all correction terms can be postponed to the end of the computation.

3.2.1 Clifford-ladder circuit

A similar argument holds when looking at *Clifford-ladder circuits*.

Definition 3.7 (Clifford-ladder circuit)**.** Let $\{U^i\}_{i=0}^{n-1}$ be a collection of n 2-qubit Clifford unitaries. A Clifford-ladder circuit C_{ladder} is a circuit of depth $\mathcal{O}(n)$ and width $\mathcal{O}(n)$ of the following form:

$$C_{ladder} = \prod_{i=0}^{n-1} U_{i,i+1}^{(i)}$$

where $U_{i,i+1}^{(i)}$ denotes that unitary $U^{(i)}$ is applied on qubits i and $i+1$.

Note that each 2-qubit Clifford unitary $U^{(i)}$ itself is of constant-depth.

The next lemma shows that any Clifford-ladder circuit has an equivalent LAQCC circuit. Figure 1 shows this mapping graphically. Each two-qubit unitary is parallelized using gate teleportation and with the Clifford commutation relations, the Pauli correction terms are pushed to the end of the computation.

Lemma 3.8. *Any Clifford-ladder circuit has an* LAQCC *implementation of depth* $\mathcal{O}(1)$ *and width* $\mathcal{O}(n)$.

Proof. Figure 1 shows the construction of a LAQCC circuit of width $\mathcal{O}(n)$ and depth $\mathcal{O}(1)$ implementing a Clifford-ladder circuit. The caps and cups denote Bell-state measurements and Bell-state creation, respectively. What remains to show is that an NC^1 circuit computes the Pauli-correction terms.

The i-th Bell measurement results in Pauli error $P_i = Z^{a_i} X^{b_i}$. A Clifford-ladder circuit of size n hence has an error vector $\begin{pmatrix} a & b \end{pmatrix}$ of length $2n$. The correction terms that have to be applied have the same form: we can label every corrective Pauli by an index j, such that $\hat{P}_j = Z^{\hat{a}_j} X^{\hat{b}_j}$. This gives a correction vector $\begin{pmatrix} \hat{a} & \hat{b} \end{pmatrix}$. Note that Pauli matrices anti-commute, hence reordering them will only incur a global phase. This implies a binary linear map $M : \begin{pmatrix} a & b \end{pmatrix} \mapsto \begin{pmatrix} \hat{a} & \hat{b} \end{pmatrix}$. As matrix vector multiplication is in NC^1, this error calculation is in NC^1 and Clifford-ladder circuits have an LAQCC implementation. □

Remark 3.9. *Constructing the binairy linear map* M *is not in* NC^1*, but it does follow directly from the quantum circuit. Instead, an* L *(logspace) precomputation gives the matrix associated to* M*.*

This result directly implies that in LAQCC we can apply two-qubit gate on any two any non-adjacent qubits.

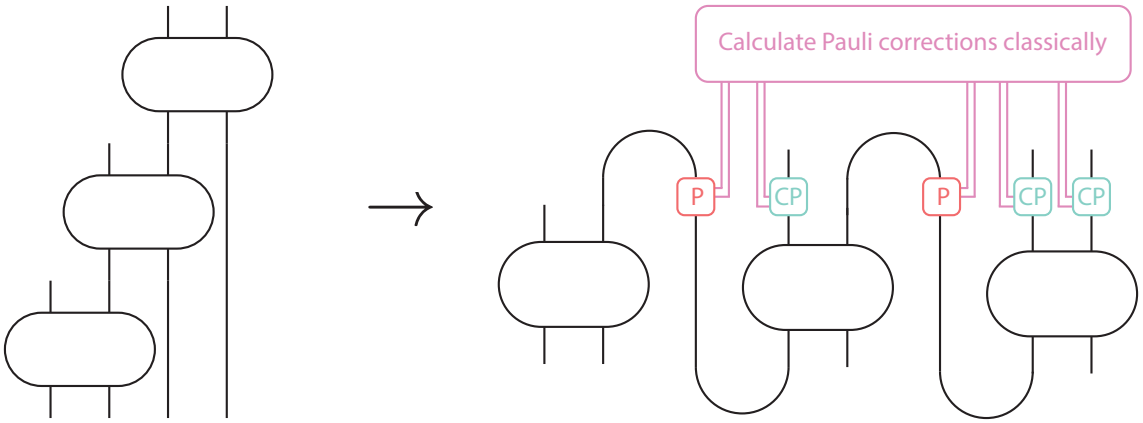


Figure 1: Graphical representation of Clifford-ladder circuit parallelization. Time flows upward and lines represent qubits and boxes quantum gates. A half circle represents either a Bell-state creation (ends pointing upwards) or a Bell-state measurement (ends pointing downwards). The Bell-state measurements can produce Pauli errors $P = Z^a X^b$, which are corrected by the boxes CP (corrective Pauli). The computations to determine how errors propagate are performed classically before the computations.

Corollary 3.10. *Any SWAP circuit needed to do an operation between non-adjacent qubits is a Clifford-ladder circuit and hence in* LAQCC.

This effectively removes the locality constraint in LAQCC *for applying a single* 2*-qubit gate on non-adjacent qubits.*

An example of a Clifford-ladder circuit is the creation of a GHZ state. We can parallelize this directly, for instance following the poor man's cat state approach of [Wat+19]. Figure 2 shows a LAQCC circuit using $2n-1$ qubits placed on a line that prepares an n-qubit GHZ state.

3.2.2 Clifford-grid circuit

Any Clifford unitary can be mapped to a linear-depth circuit given a linear nearest-neighbor architecture [MR18]. The most general representation of these circuits are so-called Clifford-grid circuits.

Definition 3.11 (Clifford-grid circuit)**.** Let n be the number of qubits. A Clifford-grid circuit of depth d is a circuit of the form

$$C_{grid} = \prod_{i=0}^{d} \bigotimes_{j=0}^{\frac{n}{2}} U_{i,j},$$

for Clifford unitaries $U_{i,j}$ and such that gate $U_{i,j}$ acts on qubits $2j$ and $2j+1$ if i is even, and $2j+1$ and $2j+2$ is i is odd.

The next lemma shows that Clifford-grid circuits also have an efficient LAQCC implementation.

Lemma 3.12. *Any Clifford-grid circuit of depth* $\mathcal{O}(n)$ *has an* LAQCC *implementation of depth* $\mathcal{O}(1)$ *and width* $\mathcal{O}(n^2)$.

Proof. Similar to the Clifford-ladder circuits, gate teleportation allows parallelization to obtain a LAQCC circuit. With a total of $\mathcal{O}(n^2)$ Clifford gates, this also requires $\mathcal{O}(n^2)$ qubits. Figure 3 illustrates the transformation.

Any Bell measurement in the circuit can incur a Pauli error, which has to be dealt with at the end of the circuit. The number of Pauli gates now scales with $\mathcal{O}(n^2)$. Similar to the Clifford-ladder circuits, there now is a vector (ab) of length $\mathcal{O}(n^2)$ containing the information of the Pauli errors. The vector of correction terms, the vector $(\hat{a}\hat{b})$, has length $\mathcal{O}(n)$.

As these Pauli errors anti-commute, there again is a binary linear map $M : (ab) \mapsto (\hat{a}\hat{b})$. The corresponding matrix is rectangular and the error-correction calculations are in NC^1. □

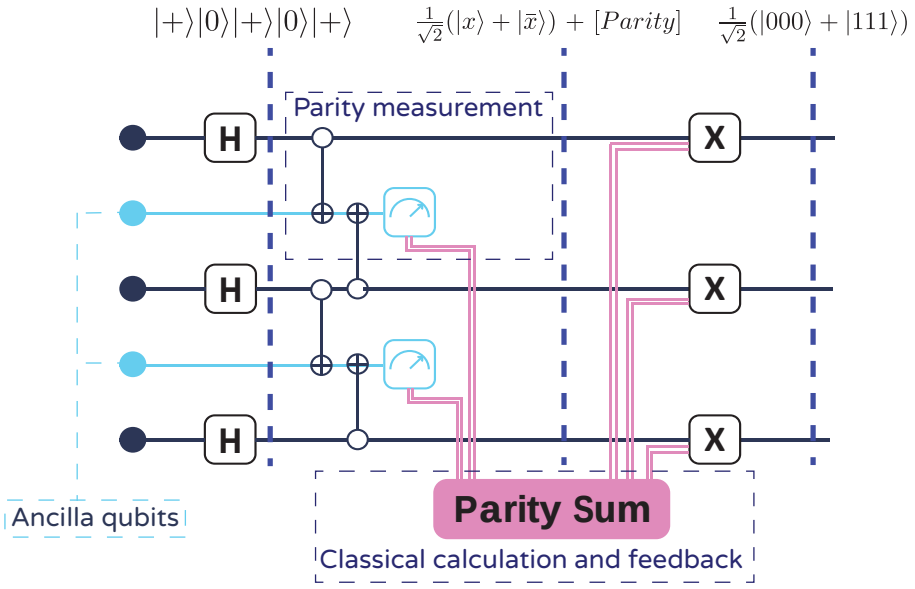


Figure 2: The quantum circuit to prepare the 3-qubit GHZ state. Double lines indicate classical information and dotted lines the quantum state at various points.

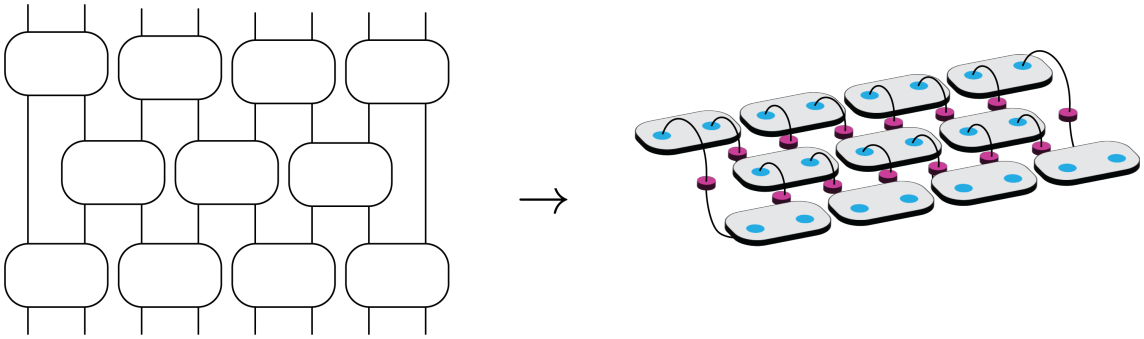


Figure 3: Graphical representation of Clifford-grid circuit parallelization. Every blue dot represents a qubit and all Clifford gates (boxes) are applied in parallel. The lines again represent Bell-state creations and Bell-state measurements, indicated by the pink boxes. The propagating Pauli errors can be corrected using the Bell-state measurement results.

Finding the matrix M for correcting a Clifford-grid circuit is more complex than for a Clifford-ladder circuit. An error occurring in the grid can have multiple paths contributing to a single output wire. For the final correction, the parity of each contributing error-path is needed. This computation is in $\oplus\mathsf{L} \subseteq \mathsf{NC}^2$ [2]. A precomputation again gives the matrix corresponding to the bilinear map M.

3.3 Useful gates and routines with an LAQCC implementation

This section groups useful multi-qubit gates with an LAQCC implementation. The construction of W-states and Dicke states uses these gates, but their use might be of broader interest.

The tables give the action of the gates on computational basis states. Their effect on arbitrary states follows by linearity. The tables also give the width of the implementation and a reference to the implementation.

The first two gates directly follow from the Clifford-parallelization results described in Section 3.2.

[2]This is not too surprising as simulating Clifford circuits classically is a complete problem for $\oplus\mathsf{L}$

Gate	Operation on basis states	Width	Implementation
Fanout_n	$\|x\rangle \|y_1\rangle \dots \|y_n\rangle \mapsto \|x\rangle \|y_1 \oplus x\rangle \dots \|y_n \oplus x\rangle$	$\mathcal{O}(n)$	Clifford-ladder circuit 3.8
$\text{Permutation}(\pi)_n$	$\|y_1\rangle \dots \|y_n\rangle \mapsto \|y_{\pi(1)}\rangle \dots \|y_{\pi(n)}\rangle$	$\mathcal{O}(n^2)$	Clifford-grid circuit 3.12

Table 2: Operations contained in LAQCC based on Clifford parallelization. Here $\pi \in S_n$ denotes a permutation of n elements.

Prior works extensively studied the fanout gate, for instance to construct a constant-depth OR_n function with one-sided error [HŠ05] and with an exact implementation [TT13, Theorem 1], both assuming the fanout gate to be a native gate. The OR_n gate also implies two other gates, as the following table shows.

Gate	Operation on basis states	Width	Implementation
OR_n	$\|y_1\rangle \dots \|y_n\rangle \|x\rangle \mapsto \|y_1\rangle \dots \|y_n\rangle \|\text{OR}_n(y) \oplus x\rangle$	$\mathcal{O}(n \log(n))$	Appendix B.1
AND_n	$\|y_1\rangle \dots \|y_n\rangle \|x\rangle \mapsto \|y_1\rangle \dots \|y_n\rangle \|\text{AND}_n(y) \oplus x\rangle$	$\mathcal{O}(n \log(n))$	negate input and output of OR_n
Equal_i	$\|j\rangle \|b\rangle \mapsto \begin{cases} \|j\rangle \|1 \oplus b\rangle & \text{if } \|j\rangle = \|i\rangle \\ \|j\rangle \|b\rangle & \text{else} \end{cases}$	$\mathcal{O}(n \log(n))$	negate part of input of AND_n

Table 3: Operations contained in LAQCC based on Fanout and local 1-qubit unitaries.

With these unbounded-fan-in OR and AND gates, all AC^0 circuits can be implemented. The next step is implementing LAQCC-type modular addition circuits, which gives circuits to check for equality and greater-than. These three gates take n-qubit quantum states as input. We introduce the indicator variable $\mathbb{1}_A$ for a Boolean expression A, which evaluates to 1 if A is true. Similarly, $|\mathbb{1}_A\rangle = |1\rangle$ if and only if A is true.

Gate	Operation on n-qubit integers $\|x\rangle$, $\|y\rangle$	Width	Implementation
Add_n	$\|x\rangle \|y\rangle \mapsto \|x\rangle \|y + x \bmod 2^n\rangle$	$\mathcal{O}(n^2)$	AC^0 circuit
Equality	$\|x\rangle \|y\rangle \|0\rangle \mapsto \|x\rangle \|y\rangle \|\mathbb{1}_{x=y}\rangle$	$\mathcal{O}(n^2)$	Appendix B.2
Greatherthan	$\|x\rangle \|y\rangle \|0\rangle \mapsto \|x\rangle \|y\rangle \|\mathbb{1}_{x>y}\rangle$	$\mathcal{O}(n^2)$	Appendix B.3

Table 4: Operations contained in LAQCC based on AC^0 circuits.

Høyer and Špalek showed that fanout-gates imply efficient constant-depth implementations of for instance the quantum Fourier transform [HŠ05]. They use this constant-depth quantum Fourier transform to construct a constant-depth circuit for weighted counting. In particular, this circuit can be used to calculate the Hamming weight of an n-bit string, and to implement an "Exact t"-gate and a threshold gate. Appendix B.5 also explains how to modify the threshold gate to a weighted threshold gate.

Remark 3.13. *As the Threshold$_t$ gate is in* LAQCC*, any classical* TC_0 *circuit is in* LAQCC*.*

This section concludes not with a gate, but with a tool used for preparing uniform superpositions. This lemma extends Lemma 2.7 to preparing states instead of finding marked items.

Lemma 3.14. *Given an n-qubit unitary U, that is implementable by a constant-depth circuit, a basis $\mathcal{C}$ and a partition of $\mathcal{C}$ in $\mathcal{G}$ and $\mathcal{B}$ such that $\frac{|\mathcal{G}|}{|\mathcal{C}|}$ is a known constant c. Suppose that U implements the map*

$$U : |y\rangle |b\rangle \mapsto \begin{cases} |y\rangle |b \oplus 1\rangle & \text{if } y \in \mathcal{G} \\ |y\rangle |b\rangle & \text{if } y \in \mathcal{B} \end{cases}.$$

Then there exists a LAQCC *circuit that prepares the state $\frac{1}{\sqrt{|\mathcal{G}|}} \sum_{y \in \mathcal{G}} |y\rangle$ by using U a constant number of times.*

Proof. Define $|\mathcal{G}\rangle = \frac{1}{\sqrt{|\mathcal{G}|}} \sum_{y \in \mathcal{G}} |y\rangle$ and $|\mathcal{B}\rangle = \frac{1}{\sqrt{|\mathcal{B}|}} \sum_{y \in \mathcal{B}} |y\rangle$. As $\mathcal{B}$ and $\mathcal{G}$ partition $\mathcal{C}$, it follows that $\langle \mathcal{G} | \mathcal{B} \rangle = 0$. Lemma 2.7 implies the existence of a circuit that prepares the desired state. Below, we explicitly construct the circuit.

Gate	Operation on n-qubit basis state $\|x\rangle$	Width	Implementation
QFT	$\|x\rangle \mapsto \frac{1}{\sqrt{2^{n-1}}} \sum_{j=0}^{2^{n-1}} e^{i2\pi \frac{x \cdot j}{2^n}} \|j\rangle$	$\mathcal{O}(n^3 \log(n))$	[HŠ05, Theorem 4.12]
Hammingweight	$\|x\rangle_n \|0\rangle_{\log(n)} \mapsto \|x\rangle_n \|\|x\|\rangle_{\log(n)}$	$\mathcal{O}(n \log(n))$	[TT13, Lemma 4]
Exact_t	$\|x\rangle \|0\rangle \mapsto \|x\rangle \left\|\mathbb{1}_{\|x\|=t}\right\rangle$	$\mathcal{O}(n \log(n))$	Appendix B.4
Threshold_t	$\|x\rangle \|0\rangle \mapsto \|x\rangle \left\|\mathbb{1}_{\sum_i x_i \geq t}\right\rangle$	$\mathcal{O}(tn \log(n))$	Appendix B.5

Table 5: Quantum subroutines in LAQCC based on Høyer and Špalek.

First, prepare a uniform superposition $\sum_{i=0}^{2^n-1} |i\rangle$. Then, iteratively reflect over the state $|\mathcal{B}\rangle$ using U, and reflect over the uniform superposition state $\sum_{i=0}^{2^n-1} |i\rangle$. Both reflections have a LAQCC implementation and we only need to apply them a constant number of iterations.

To reflect over the uniform superposition, we have to implement the operation $2 |s\rangle \langle s| - I$, with $|s\rangle = \frac{1}{\sqrt{N}} \sum_{i=0}^{2^n-1} |i\rangle$. To implement this operation, we first apply a layer of Hadamards, which implements a basis transformation mapping the uniform superposition state to the all zeroes state; Then apply the Exact_0-gate producing an output qubit that marks only the all zeroes-state and finally negate the output qubit and applies a Z-gate on it. Running this circuit in reverse, excluding the Z-gate, resets the output qubit and reverts the basis transformation. The last step of Lemma 3.14 requires a reflection using an R_Z-gate (rotational Z-gate) instead of the Z-gate. As the Exact_0-gate has an LAQCC implementation (see Table 5), this second inversion operation has a LAQCC implementation.

The total number of iterations is $\mathcal{O}(\sqrt{N/m})$, where $N = |\mathcal{C}|$ and $m = |\mathcal{G}|$. As their fraction is the constant c, it follows that $\mathcal{O}(\sqrt{c}) = \mathcal{O}(1)$ iterations are needed. □

3.4 Non-simulatability of LAQCC

Most of the power of LAQCC circuits seems to come from the classical intermediate calculations, which makes one wonder if these circuits are classically simulatable. Even if these circuits were indeed efficiently simulatable, they still have value as "fast" alternatives for state preparation. However, it is unlikely that all LAQCC circuits can be simulated efficiently by a classical simulator. Lemma 2.8 and the inclusion of the fan-out gate in LAQCC show that circuits consisting of commuting gates have an LAQCC implementation and in particular, the class of Instantaneous Quantum Polynomial-time (IQP) circuits, first introduced in [SB09], has equivalent LAQCC implementations.

Definition 3.15 (Definition 2 [NM14])**.** An IQP circuit on n qubits is a quantum circuit with the following structure: each gate in the circuit is diagonal in the Pauli-Z basis, the input state is $|+\rangle^{\otimes n}$, and the output is the result of a measurement in the Pauli-X basis on a specified set of output qubits.

Lemma 3.16. *Any* IQP *circuit has an* LAQCC *implementation.*

Proof. The following LAQCC circuit prepares the desired state: First prepare $|+\rangle^{\otimes n}$ by a single layer of Hadamard gates on all qubits. In this basis, all gates in with respect to the Pauli-Z basis commute, and hence by Lemma 2.8, we can parallelize all gates using $\text{poly}(n)$ ancilla qubits. Next, we can again apply a layer of Hadamard gates and finally measure the desired qubits. □

Bremner, Jozsa, and Shepherd showed that efficient weak classical simulation of all possible IQP circuits up to small multiplicative error implies a collapse of the polynomial hierarchy [BJS10]. Note that a circuit family is weakly simulatable if given the description of the circuit family, its output distribution can be sampled by purely classical means in $\text{poly}(n)$ time.

Lemma 3.17 (Corollary 1 [BJS10])**.** *If the output probability distributions generated by uniform families of* IQP *circuits could be weakly classically simulated to within multiplicative error* $1 \leq c < \sqrt{2}$ *then the polynomial hierarchy would collapse to the third level, in particular,* $\mathsf{PH} = \Delta_3^p$.

Corollary 3.18. *If the output probability distributions generated by uniform families of* LAQCC *circuits could be weakly classically simulated to within multiplicative error* $1 \leq c < \sqrt{2}$ *then the polynomial hierarchy would collapse to the third level, in particular,* $\mathsf{PH} = \Delta_3^p$.

3.5 LAQCC containment in QNC^1

Let A be an LAQCC-circuit. We can write this circuit as a composition of unitary quantum layers U_i, measurements M_i and classical calculation layers C_i:

$$A = M_k U_k C_k \dots M_i U_i C_i \dots M_1 U_1 C_1,$$

for some constant k. Any unitary U_i is a QNC^0 circuit and any C_i is an NC^1-circuit. The measurements M_i can measure any subset of the qubits. By the principle of deferred measurements, we can always postpone them to the end of the circuit using $CNOT$ gates and fresh ancilla qubits [NC10, Section 4.4], which gives the following lemma.

Lemma 3.19. *For any* LAQCC*-circuit A there is a* QNC^1*-circuit B without intermediate measurements that outputs the same state as A.*

Proof. The LAQCC-circuit A contains classical computation layers C_i that use the intermediate measurement results as input. These measurements can be delayed until the end of the circuit by applying a CNOT from the qubit to a fresh ancilla qubit. This replaces the classical output wires by quantum wires.

Lemma A.1 shows that any NC^1-circuit can be run on a log-depth quantum circuit with $\mathcal{O}(\mathrm{poly}(n))$ ancilla qubits. Hence, a QNC^1-circuit without topology constraints can take the role of the classical intermediate circuits C_i.

Now, let V_i be the quantum circuit implementing C_i. Then the quantum circuit

$$B = U_k V_k \dots U_1 V_1$$

is a quantum circuit of logarithmic depth simulating A. □

3.6 Complexity results for $\mathsf{LAQCC}(\mathcal{Q}, \mathcal{C}, d)$

In the definition of $\mathsf{LAQCC}(\mathcal{Q}, \mathcal{C}, d)$, we have freedom to choose $\mathcal{Q}$ and $\mathcal{C}$. If we give more power to both the quantum and the classical routines, we see that we can solve more complex problems and prepare a wider variety of quantum states. Yet, even with polynomial depth quantum circuits and unbounded classical computational power, limits exist.

Notation 3.20. The class LAQCC^* is the instantiation $\mathsf{LAQCC}(\mathsf{QPoly}(n), \mathsf{ALL}, \mathrm{poly}(n))$: The class of polynomially many alternating polynomial-sized quantum circuits and arbitrary powerful classical computations, together with feed-forward of the classical information to future quantum operations. The quantum computations are restricted to all single-qubit gates and the two-qubit CNOT gate.

This directly gives us a definition of $\mathsf{StateLAQCC}^*_\varepsilon$ for $\varepsilon > 0$.

Remark 3.21. *Note that for any non-zero ε, we can restrict ourselves to finite universal gate-sets. The Solovay-Kitaev theorem [Kit97; NC10] says that any multi-qubit unitary can be approximated to within precision δ by a quantum circuit with size depending on δ. Therefore, with a finite universal gate-set, any* LAQCC^* *circuit with a continuous gate-set can be approximated by an* LAQCC^* *circuit with gates from the finite set.*

Next, we prove $\mathsf{StateLAQCC}^*_\varepsilon \subseteq \mathsf{StatePostQPoly}_\varepsilon$. Following the same decomposition as in the previous section, we find that any LAQCC^* can be written as

$$\Pi_{i=0}^{\mathrm{poly}(n)} M_i U_i(y_i) C_i(x_i) \left|0\right\rangle^{\otimes \mathrm{poly}(n)}, \tag{1}$$

where again, M_i denotes the i-th measurement layer, U_i the i-th quantum layer and C_i the i-th unbounded classical computation layer. The $x_i \in \{0,1\}^*$ denote the outcomes of M_i and $y_i \in \{0,1\}^*$ the bitstring outputted by C_i. Note, all x_i and y_i have length at most polynomial in n.

Theorem 3.22. *It holds that* $\mathsf{StateLAQCC}^*_\varepsilon \subseteq \mathsf{StatePostQPoly}_\varepsilon$.

Proof. Fix $\varepsilon > 0$ and a positive integer n and let $|\psi\rangle \in \mathsf{StateLAQCC}^*_\varepsilon$. By definition, there exists an LAQCC^* circuit $A = \Pi_{i=0}^{\mathrm{poly}(n)} M_i U_i(y_i) C_i(x_i)$, which prepares a state $|\phi\rangle$ with inner product at least $1-\varepsilon$ with $|\psi\rangle$.

Then consider the following PostQPoly-circuit: Let $B = \Pi_{i=0}^{\mathrm{poly}(n)} \mathrm{Equal}_{x_i}(x_i) U_i(y_i) |0\rangle^{\otimes \mathrm{poly}(n)}$, where the y_i are hardwired. The Equal_{x_i} gate replaces the measurement layer M_i, by checking if the subset of qubits that would be measured are in $|x_i\rangle$ computational basis state. It stores the output in an ancilla qubit. As a last step, apply an $\mathrm{AND}_{\mathrm{poly}(n)}$-gate on the ancilla qubits, which hold the outputs of the Equal_{x_i} gates, and store the result in an ancilla qubit. Conditional on this last ancilla qubit being one, the circuit prepares the state $|\phi\rangle$. □

Figure 4 gives a schematic overview of the proof and the translation of an LAQCC^* circuit in a PostQPoly-circuit.

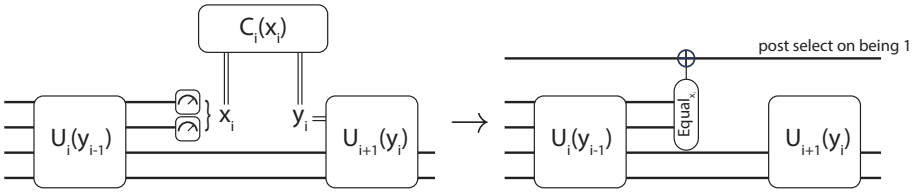


Figure 4: Schematic idea of transforming an LAQCC^* circuit for generating $|\psi\rangle$ into a PostQPoly circuit.

Further complexity studies of LAQCC: An interesting direction for future work is to extend the inclusion proved above to a true separation or an oracular separation. One approach is to use a similar oracle as used in [AK07] to separate QMA and QCMA with respect to an oracle and use a counting argument to argue that $\mathsf{StateLAQCC}^{*\mathcal{O}}_\varepsilon \neq \mathsf{StatePostQPoly}^{\mathcal{O}}_\varepsilon$, for some oracle $\mathcal{O}$ and $\varepsilon = 1 - \frac{1}{\mathrm{poly}(n)}$.

Another interesting direction for studying the state complexity of LAQCC is in the fact that the LAQCC model allows for a constant number, more than one, of rounds of measurements and corrections. This was required for our three new state generation protocols. However other models considered only one round of measurements and corrections, for instance in the paper [PSC21]. One may wonder if there is a hierarchy of model power allowing one or multiple measurements, and if there is a way to reduce the number of measurements rounds. A starting effort towards classifying types of states based on such a hierarchy can be found in [TVV23b]. It would be interesting to see a more extensive complexity theoretic analysis comparing models with different number of allowed rounds.

4 State preparation in LAQCC

In this section we consider what quantum states we can prepare using an LAQCC circuit beyond the stabilizer states and Clifford circuits discussed in the previous section. Specifically, as mentioned in the introduction, we consider quantum states that are widely used, in other quantum algorithms, for bench marking purposes and within physics. First, we show how to create a uniform superposition of computational basis states up to size q, where q is not a power of 2, a state that is often used as initial state in other algorithms (including the following other state preparation protocols presented in this work). We then use this procedure to create W-states, the uniform superposition over all n-bitstrings of Hamming-weight 1, using a compress-uncompress method. This compress-uncompress method generalizes to preparing Dicke-(n,k) states for $k = \mathcal{O}(\sqrt{n})$, uniform superpositions over all n-bitstrings of Hamming-weight $k = \mathcal{O}(\sqrt{n})$. Dicke states find many applications, and especially the compress-uncompress approach might prove useful for entanglement distillation protocols. Preparing general Dicke-(n,k) states requires a novel method to map between two integer representation systems, the factoradic representation and the combinatorial number representation. Finally, we present a state preparation protocol for quantum many-body scar states, states often used in physics, based on the Dicke-(n,k) state preparation protocol for $k = \mathcal{O}(\sqrt{n})$

4.1 Uniform superposition of size q

The uniform superposition is often used as initial state in other quantum algorithms. A simple Hadamard gate applied to n qubits prepares the uniform superposition $\frac{1}{\sqrt{2^n}}\sum_{i=0}^{2^n-1}|i\rangle$. Preparing the state $\frac{1}{\sqrt{q}}\sum_{i=0}^{q-1}|i\rangle$, the superposition up to size q, is already harder for arbitrary q.

A simple probabilistic approach works as follows: 1) create a superposition $\frac{1}{\sqrt{2^n}}\sum_{i=0}^{2^n-1}|i\rangle$ with $n = \lceil \log_2(q) \rceil$ qubits; 2) mark the states $i < q$ using an ancilla qubit; 3) measure this ancilla qubit. Based on the measurement result, the desired superposition is found, which happens with probability at least one half.

The next theorem modifies this probabilistic approach to a protocol that deterministically prepares the uniform superposition modulo q in LAQCC.

Theorem 4.1. *There is a deterministic* LAQCC *circuit that prepares the uniform superposition of size q. This circuit requires $\mathcal{O}(\lceil \log_2(q) \rceil^2)$ qubits.*

Proof. Let $n = \lceil \log_2(q) \rceil$ and define $\mathcal{G} = \{i \mid 0 \leq i < q\}$ and $\mathcal{B} = \{i \mid q \leq i \leq 2^n - 1\}$. Construct the unitary

$$U_q : |y\rangle|b\rangle \mapsto \begin{cases} |y\rangle|b \oplus 1\rangle & \text{if } y < q \\ |y\rangle|b\rangle & \text{if } y \geq q \end{cases}.$$

The Greaterthan-gate of Table 4 implements the operator U_q, note that this gate requires $\mathcal{O}(n^2)$ qubits.

As $|\mathcal{G}|/2^n \geq 1/2$ and known, applying Lemma 3.14 with the sets $\mathcal{G}$ and $\mathcal{B}$ and the constant-depth implementation of U_q, gives a LAQCC algorithm that boosts the amplitude of $|\mathcal{G}\rangle$ to 1. □

Remark 4.2. *Note that in Lemma 3.14 it was implicitly assumed that $|\mathcal{G}| + |\mathcal{B}|$ is a power of two (allowing for a simple reflection over the uniform superposition state). This* LAQCC *implementation of creating a uniform superposition modulo any q removes this requirement.*

4.2 W-state in LAQCC

In this section we consider the W_n-state and how to prepare this state in LAQCC. The W_n-state is a uniform superposition over all n-qubit states with a single qubit in the $|1\rangle$-state and all others in the $|0\rangle$-state:

$$|W_n\rangle = \frac{1}{\sqrt{n}}\sum_i |e_i\rangle,$$

where $|e_i\rangle$ is the state with a one on the i-th position and zeroes elsewhere.

A first observation is that the W-state can be seen as a one-hot encoding of a uniform superposition over n elements. We can label the n states with non-zero amplitude of the W-state with an index. More precisely, we want to find circuits that implement the following map:

$$|i\rangle|0\rangle \mapsto |0\rangle|e_i\rangle, \tag{2}$$

with i an index and e_i the one-hot encoding of i. This index – which equals the position of the 1 – compresses the representation from n to $\log(n)$ bits. This compression naturally defines two operations:

$$\textbf{Uncompress:}\quad |i\rangle_{\log(n)}|0\rangle_n \mapsto |i\rangle_{\log(n)}|e_i\rangle_n, \tag{3}$$

$$\textbf{Compress:}\quad |i\rangle_{\log(n)}|e_i\rangle_n \mapsto |0\rangle_{\log(n)}|e_i\rangle_n. \tag{4}$$

Implementing both and combining them implements Mapping 2 giving an efficient W-state preparation protocol.

The **Compress** and **Uncompress** operations map between a one-hot and binary representation of an integer i. We call the registers containing the binary representation index registers, and the register containing the one-hot representation the system register. The index registers serve as ancilla qubits and the W-state is prepared in the system register.

Lemma 4.3. *There exists an* LAQCC *circuit that, for any n, implements the* ***Uncompress*** *operation:*

$$\frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle_{\log(n)}|0\rangle_n \mapsto \frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle|e_i\rangle_n\,.$$

This circuit uses $\mathcal{O}(n\log(n))$ qubits placed in a grid pattern of size $n\times\log(n)$.

Proof. One column of the grid of length n consists of system qubits placed in a line. Adjacent to this line are $\log(n)$ index qubits. The left grid in Figure 5 shows the initial layout. The same figure also shows the steps to prepare the W-state in the system qubits.

$$\begin{aligned}\frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle_{\log(n)}|0\rangle_{\log(n)}^{\otimes n-1}|0\rangle_n &\xrightarrow{(1)} \frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle_{\log(n)}^{\otimes n}|0\rangle_n\\ &\xrightarrow{(2)} \frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle_{\log(n)}^{\otimes n}|e_i\rangle_n\\ &\xrightarrow{(3)} \frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle|0\rangle^{\otimes n-1}|e_i\rangle_n\end{aligned}$$

Step (1) uses fanout-gates to create a fully entangled state between the different index registers. Step (2) applies Equal_i gates in parallel from each index register to its corresponding system qubit to create the state $|e_i\rangle$ in the system register. Step (3) uses fanout-gates to disentangle and reset the index registers. Combined the **Uncompress** operation maps $\frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle_{\log(n)}|0\rangle_{\log(n)}^{\otimes n-1}|0\rangle_n \mapsto \frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle|0\rangle^{\otimes n-1}|e_i\rangle_n$ as required. □

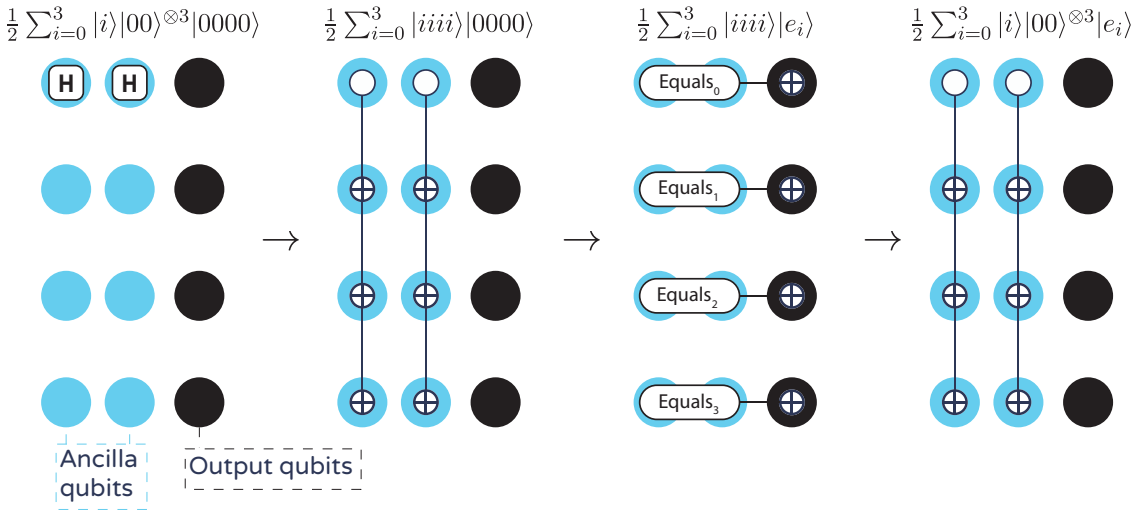


Figure 5: Circuit for the **Uncompress** operation for $n=4$. Shown is a grid of 12 qubits: 8 blue index qubits, and 4 black system qubits. Each of the four grids represents a single timeslice in the **Uncompress** operation.

Lemma 4.4. *There exists an* LAQCC *circuit that, for any n, implements the* ***Compress*** *operation:*

$$\frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle_{\log(n)}|e_i\rangle_n \mapsto \frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|0\rangle|e_i\rangle_n\,.$$

This circuit uses $\mathcal{O}(n\log(n))$ qubits placed in a grid pattern of size $n\times\log(n)$.

Proof. To implement **Compress**, the index registers are uncomputed using parallel $CNOT$-operations, controlled by the system register. These controlled gates commute for different indices

in the system register and hence by Lemma 2.8 a parallel circuit for the uncomputation exists. The **Compress** operation, also shown in Figure 6, consists of the operations:

$$\begin{aligned}
\frac{1}{\sqrt{n}}\sum_{i=0}^{n}|i\rangle_{\log(n)}|0\rangle_{\log(n)}^{\otimes n-1}|e_i\rangle_n &\xrightarrow{(1)} \frac{1}{n}\sum_{i,j=0}^{n}(-1)^{i\cdot j}|j\rangle_{\log(n)}|0\rangle_{\log(n)}^{\otimes n-1}|e_i\rangle_n \\
&\xrightarrow{(2)} \frac{1}{n}\sum_{i,j=0}^{n}(-1)^{i\cdot j}|j\rangle_{\log(n)}^{\otimes n}|e_i\rangle_n \\
&\xrightarrow{(3)} \frac{1}{n}\sum_{i,j=0}^{n}|j\rangle_{\log(n)}^{\otimes n}|e_i\rangle_n \\
&\xrightarrow{(4)} \frac{1}{n}\sum_{i,j=0}^{n}|j\rangle_{\log(n)}|0\rangle_{\log(n)}^{\otimes n-1}|e_i\rangle_n \\
&\xrightarrow{(5)} \frac{1}{\sqrt{n}}\sum_{i=0}^{n}|0\rangle_{\log(n)}^{\otimes n}|e_i\rangle_n
\end{aligned}$$

Step (1) applies Hadamard gates to the first index register, changing from the computational to the Hadamard basis, in which the NOT-operation is diagonal; Step (2) uses fanout-gates to create a fully entangled state in the index registers; Step (3) applies controlled-Z gates, controlled by the system qubit i and with targets the qubits in the i-th index register corresponding to the ones in the binary representation of i; Step (4) disentangles the index registers using fanout-gates; and, Step (5) applies Hadamard gates to clean the index register.

The controlled-Z gates in Step (3) apply phases that precisely cancel the phases already present, which disentangles the index registers from the system register. □

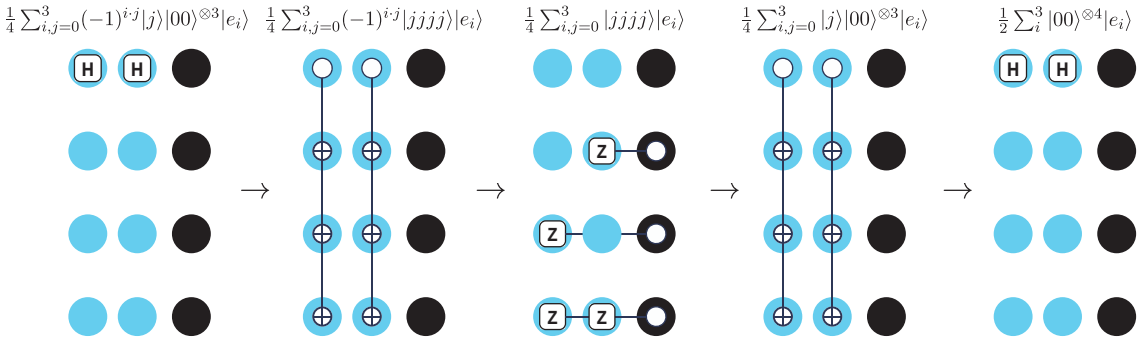


Figure 6: Circuit for the **Uncompress** operation for $n = 4$. Shown is a grid of 12 qubits: 8 blue index qubits, and 4 black system qubits. Each of the four grids represents a single timeslice in the **Compress** circuit.

Theorem 4.5. *There exists a circuit in* LAQCC *that prepares the* $|W_n\rangle$ *state. This circuit requires* $\mathcal{O}(n\log(n))$ *qubits placed in a grid of size* $n \times \log(n)$.

Proof. The circuit combines the circuits of Theorem 4.1, Lemma 4.3 and Lemma 4.4. It consists of three steps:

$$\begin{aligned}
|0\rangle_{\log(n)}^{\otimes n}|0\rangle_n &\xrightarrow{(1)} \frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle|0\rangle^{\otimes n-1}|0\rangle \\
&\xrightarrow{(2)} \frac{1}{\sqrt{n}}\sum_{i=0}^{n-1}|i\rangle|0\rangle^{\otimes n-1}|e_i\rangle \\
&\xrightarrow{(3)} \frac{1}{\sqrt{n}}\sum_{i=0}^{n}|0\rangle^{\otimes n}|e_i\rangle
\end{aligned}$$

Step one prepares the uniform superposition over indices, this can be done either by applying a layer of Hadamard gates, if n is a power of 2, requiring $\mathcal{O}(log(n))$ qubits, or using Theorem 4.1 if n is not a power of 2 requiring $\mathcal{O}(\lceil \log(n) \rceil^2)$ qubits; Step (2) is by Lemma 4.3 and requires $\mathcal{O}(n \log(n))$ qubits; and, Step(3) is by Lemma 4.4 and requires $\mathcal{O}(n \log(n))$ qubits. □

4.3 Dicke states for small k

In this section we generalize our method of preparing the $|W\rangle$-state in Theorem 4.5 to a more general set of states, Dicke states. A Dicke-(n, k) state is the uniform superposition over bitstrings of Hamming weight k and length n (which we again assume to be a power of 2 for simplicity):

$$|D_k^n\rangle = \binom{n}{k}^{-1/2} \sum_{x \in \{0,1\}^n : |x|=k} |x\rangle \,. \tag{5}$$

For $k = 1$, this state is precisely the W-state. There exists an efficient deterministic method to prepare a $|D_k^n\rangle$ state that requires a circuit of width $\mathcal{O}(n)$ and depth $\mathcal{O}(n)$, independent of k [BE19]. This methods starts from the $|1\rangle^{\otimes k} |0\rangle^{\otimes k-n}$ state and relies on a recursive formula for the Dicke state

$$|D_k^n\rangle = \alpha_{k,n} \left|D_k^{n-1}\right\rangle \otimes |0\rangle + \beta_{k,n} \left|D_{k-1}^{n-1}\right\rangle \otimes |1\rangle \,.$$

This relation implies a protocol that is inherently sequential, which is unsuited for an LAQCC implementation.

Instead, we present an LAQCC approach similar to the W-state preparation protocol. We apply the **Uncompress** operation of the W-state in parallel to put k ones into the bitstring. This method allows for the preparation of Dicke states with $k = \mathcal{O}(\sqrt{n})$, using $\mathcal{O}(n^2 \log(n)^3)$ qubits. The bound on k comes from the fact that using the **Uncompress** operation in parallel might cause overlaps to where the 1's are put into the system register. Having two ones in the same system qubit in effect negates the **Uncompress** operation. Following the lines of the birthday paradox, we find that overlaps between different indices happen not that often for $k = \mathcal{O}(\sqrt{n})$. Lemma 3.14 allows us to boost the amplitudes and make the protocol deterministic.

Again, consider two groups of qubits: Index registers with $\log(n)$ qubits each; and, system registers of n qubits each. Contrary to the W-state, the Dicke state requires multiple system registers during the preparation. The state is prepared in only one system register. Denote the index registers with subscripts i_1 up to i_k and the system registers with s_1 up to s_n. For clarity, these indices may be omitted if it is clear from the context.

The algorithm consists of four steps:

1. **Filling**: $|0\rangle_{i_1} \ldots |0\rangle_{i_k} |0\rangle_{s_1} \to \frac{1}{n^{k/2}} \sum_{j_1,\ldots,j_k=0}^{n-1} |j_1\rangle_{i_1} \ldots |j_k\rangle_{i_k} |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle_{s_1}$
2. **Filtering**: $\to \sqrt{\frac{(n-k)!}{n!}} \sum_{j_1 \neq \ldots \neq j_k}^{n-1} |j_1\rangle \ldots |j_k\rangle |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle$
3. **Ordering**: $\to \frac{1}{\sqrt{\binom{n}{k}}} \sum_{j_1 < \cdots < j_k}^{n-1} |j_1\rangle \ldots |j_k\rangle |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle$
4. **Cleaning**: $\to \frac{1}{\sqrt{\binom{n}{k}}} \sum_{j_1 < \cdots < j_k}^{n-1} |0\rangle \ldots |0\rangle |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle$

Note that after **Filling** there is a multiplicity in states. First, **Filtering** removes those states in which different indices j_l are the same, resulting in an incorrect state in the s_1 register. Second, **Ordering** removes the multiplicity from having multiple permutations of the index registers creating the same state in the s_1 register, by forcing a choice of ordering on the indices. These two steps give a unique pairing between index registers and system registers allowing the operation **Cleaning**.

We will now proof that these four steps are achievable in LAQCC and explicitly visualize the corresponding circuits for $n = 4$ and $k = 2$.

Lemma 4.6. *An* LAQCC *circuits exists that implements* ***Filling****:*

$$|0\rangle_{i_1} \ldots |0\rangle_{i_k} |0\rangle_{s_1} \to \frac{1}{n^{k/2}} \sum_{j_1,\ldots,j_k=0}^{n-1} |j_1\rangle_{i_1} \ldots |j_k\rangle_{i_k} |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle_{s_1} \,.$$

This circuit uses $\mathcal{O}(kn\log(n))$ qubits.

Proof. To achieve a circuit implementing **Filling** we use **Uncompress** from Lemma 4.3 k times in parallel. Note that two **Uncompress** operations commute, hence by Lemma 2.8 k **Uncompress** operations can be implemented in parallel. Each of these parallel operations requires an index register, a system register and $\mathcal{O}(n\log(n))$ extra ancilla qubits.

The corresponding circuit consists of five steps:

$$
\begin{aligned}
|0\rangle_{i_1}\dots|0\rangle_{i_k}|0\rangle_{s_1}\dots|0\rangle_{s_k} &\xrightarrow{(1)} \frac{1}{n^{k/2}}\sum_{j_1\dots j_k=0}^{n-1}|j_1\rangle\dots|j_k\rangle\frac{1}{\sqrt{2^n}}\sum_{l=0}^{2^n-1}|l\rangle_{s_1}|0\rangle_{s_2}\dots|0\rangle_{s_k}\\
&\xrightarrow{(2)} \frac{1}{n^{k/2}}\sum_{j_1\dots j_k=0}^{n-1}|j_1\rangle\dots|j_k\rangle\frac{1}{\sqrt{2^n}}\sum_{l=0}^{2^n-1}|l\rangle_{s_1}|l\rangle_{s_2}\dots|l\rangle_{s_k}\\
&\xrightarrow{(3)} \frac{1}{n^{k/2}}\sum_{j_1\dots j_k=0}^{n-1}|j_1\rangle\dots|j_k\rangle\frac{1}{\sqrt{2^n}}\sum_{l=0}^{2^n-1}(-1)^{(2^{j_1}+\dots+2^{j_k})\cdot l}|l\rangle_{s_1}|l\rangle_{s_2}\dots|l\rangle_{s_k}\\
&\xrightarrow{(4)} \frac{1}{n^{k/2}}\sum_{j_1\dots j_k=0}^{n-1}|j_1\rangle\dots|j_k\rangle\frac{1}{\sqrt{2^n}}\sum_{l=0}^{2^n-1}(-1)^{(2^{j_1}+\dots+2^{j_k})\cdot l}|l\rangle_{s_1}|0\rangle_{s_2}\dots|0\rangle_{s_k}\\
&\xrightarrow{(5)} \frac{1}{n^{k/2}}\sum_{j_1\dots j_k=0}^{n-1}|j_1\rangle\dots|j_k\rangle|e_{j_1}\oplus\dots\oplus e_{j_k}\rangle_{s_1}|0\rangle_{s_2}\dots|0\rangle_{s_k}
\end{aligned}
$$

Step (1) brings all index registers in a uniform superposition of size n, use Theorem 4.1 if required, and one system register in a uniform superposition size 2^n; Step (2) uses fan-out gates to create entangled copies of the system register; Step (3) applies a phase flip between every pair of index and system register using **Uncompress** of Lemma 4.3, except instead of applying not gates to the system registers, apply phase gates; Step (4) uses fan-out gates to disentangle and uncompute all but one of the system registers; Step (5) applies Hadamard gates on the system register to obtain the one-hot representation of the index registers. Step(3), the step that requires most qubits, requires $\mathcal{O}(n\log(n))$ qubits for every pair of index and system register, of which there are k, resulting in the requirement of $\mathcal{O}(kn\log(n))$ qubits. □

Figure 7 shows these five steps graphically. Ancilla qubits are omitted for clarity. Note that some of the j_i in the index registers may intersect. The next Filtering step takes care of that.

Lemma 4.7. *An* LAQCC *circuit exists that implements* ***Filtering****:*

$$
\frac{1}{n^{k/2}}\sum_{j_1,\dots,j_k=0}^{n-1}|j_1\rangle_{i_1}\dots|j_k\rangle_{i_k}|e_{j_1}\oplus\dots\oplus e_{j_k}\rangle_{s_1}\to\sqrt{\frac{(n-k)!}{n!}}\sum_{j_1\neq\dots\neq j_k}^{n-1}|j_1\rangle\dots|j_k\rangle|e_{j_1}\oplus\dots\oplus e_{j_k}\rangle\,.
$$

Proof. First note that the state produced by the **Filling** step,

$$
\frac{1}{n^{k/2}}\sum_{j_1\dots j_k=0}^{n-1}|j_1\rangle\dots|j_k\rangle|e_{j_1}\oplus\dots\oplus e_{j_k}\rangle_{s_1}\,,
$$

contains states in which some of the indices j_l overlap. Let $|\psi\rangle=\sum_{j_1\neq\dots\neq j_k}|j_1\rangle\dots|j_k\rangle|e_{j_1}\oplus\dots\oplus e_{j_k}\rangle$, be the state in which none of the indices overlap, the desired output state. Then we can write

$$
\frac{1}{n^{k/2}}\sum_{j_1\dots j_k=0}^{n-1}|j_1\rangle\dots|j_k\rangle|e_{j_1}\oplus\dots\oplus e_{j_k}\rangle_{s_1}=\alpha|\psi\rangle+\beta|\psi^\perp\rangle\,,
$$

with $|\psi^\perp\rangle$ containing the states in which at least two of the indices j_l ovelap. Note that $\langle\psi|\psi^\perp\rangle=0$, so α can be exactly calculated by counting the number of quantum states with distinct j_i's, which gives $|\alpha|^2=\frac{n!}{(n-k)!n^k}$. Lemma A.2 gives a lower bound on $|\alpha|^2$:

$$
|\alpha|^2=\frac{n!}{(n-k)!n^k}>e^{\frac{-2k^2}{n}}\,,
$$

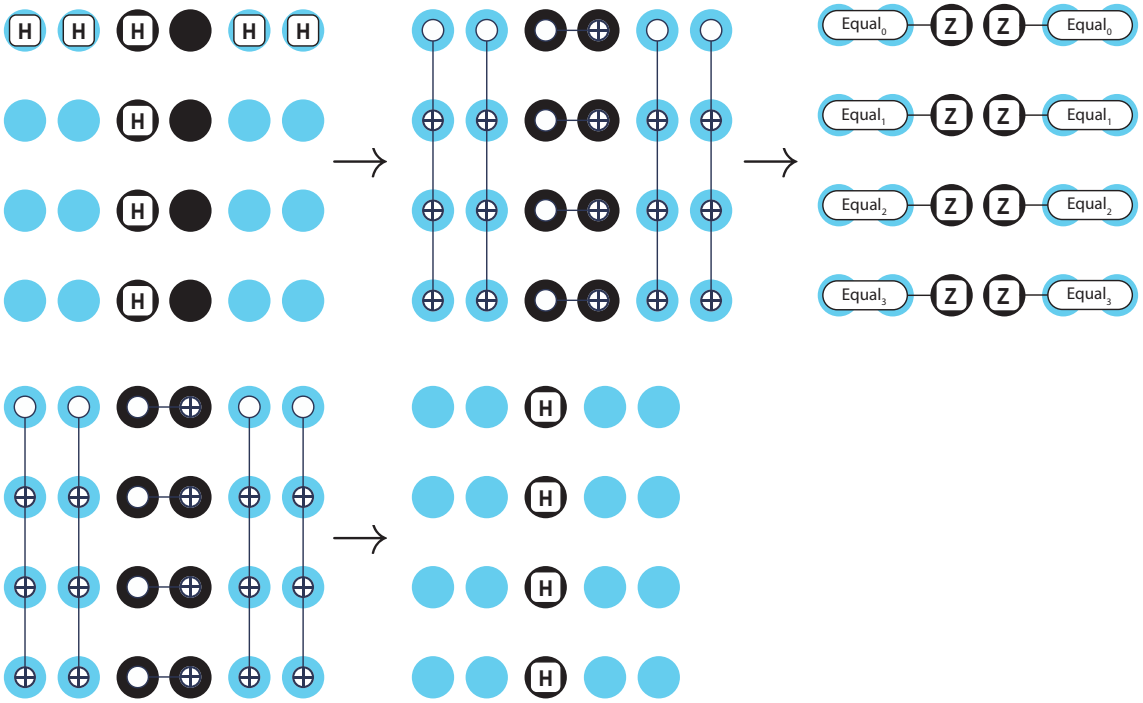


Figure 7: Circuit to implement **Filling**, $|0\rangle_{i_1} \dots |0\rangle_{i_k} |0\rangle_{s_1} \to \frac{1}{n^{k/2}} \sum_{j_1,\dots,j_k=0}^{n-1} |j_1\rangle_{i_1} \dots |j_k\rangle_{i_k} |e_{j_1} \oplus \dots \oplus e_{j_k}\rangle_{s_1}$. This circuit requires $\mathcal{O}(kn \log(n))$, for $n = 4$ and $k = 2$. A grid of 24 qubits is shown: 16 blue index qubits and 8 black system qubits. Each of the five grids represents a single timeslice in the circuit.

which is at least constant for $k = \mathcal{O}(\sqrt{n})$.

The state $|\psi^\perp\rangle$ is a superposition of states in which the system register state has Hamming weight less than k, because at least two of the j_i's are the same causing a cancellation in the system register. We can use this to create a unitary U_{flag} that flags $|\psi^\perp\rangle$. We implement this in two steps:

$$
\begin{aligned}
&\frac{1}{n^{k/2}} \sum_{j_1,\dots,j_k=0}^{n-1} |j_1\rangle \dots |j_k\rangle |e_{j_1} \oplus \dots \oplus e_{j_k}\rangle_{s_1} |0\rangle_{\log(k)} |0\rangle \\
&\xrightarrow{(1)} \frac{1}{n^{k/2}} \sum_{j_1,\dots,j_k=0}^{n-1} |j_1\rangle \dots |j_k\rangle |e_{j_1} \oplus \dots \oplus e_{j_k}\rangle_{s_1} ||e_{j_1} \oplus \dots \oplus e_{j_k}|\rangle |0\rangle \\
&\xrightarrow{(2)} \frac{1}{n^{k/2}} \sum_{j_1,\dots,j_k=0}^{n-1} |j_1\rangle \dots |j_k\rangle |e_{j_1} \oplus \dots \oplus e_{j_k}\rangle_{s_1} |0\rangle \left|\mathbb{1}_{|e_{j_1} \oplus \dots \oplus e_{j_k}|=k}\right\rangle \\
&= \alpha |\psi\rangle |1\rangle + \beta |\psi^\perp\rangle |0\rangle
\end{aligned}
$$

Where $|x|$ denotes the Hamming weight of bitstring x. Step (1) follows from a Hamming-weight gate (see Table 5), which requires $\mathcal{O}(n \log(n))$ qubits; Step (2) follows from applying an Exact$_k$ gate, requiring $\mathcal{O}(\log(n)^2)$ qubits. This same step also uncomputes the Hamming-weight gate of the first step.

Lemma 3.14 now allows us to amplify α to 1 using the oracle U_{flag}. This produces the state

$$
\sqrt{\frac{(n-k)!}{(n)!}} \sum_{j_1 \neq \dots \neq j_k} |j_1\rangle \dots |j_k\rangle |e_{j_1} \oplus \dots \oplus e_{j_k}\rangle ,
$$

using $\mathcal{O}(kn \log(n))$ qubits. □

To uncompute the index registers, we have to know which one in the system register corresponds to which index register, as any permutation of the index registers results in the same state in the

system register. The **Ordering** step imposes an ordering on the index registers, thereby removing the redundancy in the ordering.

Lemma 4.8. *An* LAQCC *circuit exists that implements* ***Ordering****:*

$$\sqrt{\frac{(n-k)!}{n!}} \sum_{j_1 \neq \ldots \neq j_k}^{n-1} |j_1\rangle \ldots |j_k\rangle |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle \to \frac{1}{\sqrt{\binom{n}{k}}} \sum_{j_1 < \cdots < j_k}^{n-1} |j_1\rangle \ldots |j_k\rangle |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle .$$

This circuit uses $\mathcal{O}(k^2 \log(n)^2)$ *qubits.*

Proof. The first step of the LAQCC circuit that implements **Ordering** is to evaluate a Greaterthan-gate on all pairs of index registers, which requires k copies of each index register. We require k extra qubits per index register to store the outcome of the Greaterthan-gates. The copies of the index registers are created by doing a fan-out gate. Note that the distribution of the index registers should be set up in such a way that every possible pair can be compared by a Greaterthan-gate.

$$\sqrt{\frac{(n-k)!}{n!}} \sum_{j_1 \neq \ldots \neq j_k} |j_1\rangle^{\otimes k} |0\rangle^{\otimes k} \ldots |j_k\rangle^{\otimes k} |0\rangle^{\otimes k} |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle \xrightarrow{(1)}$$
$$\sqrt{\frac{(n-k)!}{n!}} \sum_{j_1 \neq \ldots \neq j_k} \left[|j_1\rangle^{\otimes k} |\mathbb{1}_{j_1 > j_2}\rangle \ldots |\mathbb{1}_{j_1 > j_k}\rangle \right] \ldots \left[|j_k\rangle^{\otimes k} |\mathbb{1}_{j_k > j_1}\rangle \ldots \left|\mathbb{1}_{j_k > j_{k-1}}\right\rangle \right] |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle .$$

Each $\mathbb{1}_{j_k > j_{k'}}$ is an indicator variable that evaluates to one if and only if $j_k > j_{k'}$. This step requires $\mathcal{O}(k^2 \log(n)^2)$ qubits.

Next, we compute and measure the Hamming weight of the ancilla qubits $|\mathbb{1}_{j_1 > j_2}\rangle \ldots |\mathbb{1}_{j_1 > j_k}\rangle$, using the Hamming-weight gate. We measure the calculated Hamming weights. As all index registers were distinct before measuring, these measurements directly impose an ordering on the index registers.

$$\xrightarrow{(\text{Hammingweight})} \sqrt{\frac{(n-k)!}{n!}} \sum_{j_1 \neq \ldots \neq j_k} \left[|j_1\rangle |\mathbb{1}_{j_1 > j_2} + \cdots + \mathbb{1}_{j_1 > j_k}\rangle \right]$$
$$\ldots \left[|j_k\rangle \left|\mathbb{1}_{j_k > j_1} + \cdots + \mathbb{1}_{j_k > j_{k-1}}\right\rangle \right] |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle$$
$$\xrightarrow{(measure)} \binom{n}{k}^{-1/2} \sum_{j_1 < \cdots < j_k} \left[|j_1\rangle |0\rangle \right] \ldots \left[|j_k\rangle |k\rangle \right] |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle$$

This step costs $\mathcal{O}(k^2 \log(k))$ qubits. Assume without loss of generality that the measurement outcomes impose the ordering $j_1 < \cdots < j_k$. Otherwise, a permutation of the index registers achieves the same ordering, using the Permutation gate from Table 2.

Uncomputing the Hamming weights and the Greaterthan-gates gives the state

$$\binom{n}{k}^{-1/2} \sum_{j_1 < \cdots < j_k} \left[|j_1\rangle \ldots |j_k\rangle \right] |e_{j_1} \oplus \cdots \oplus e_{j_k}\rangle .$$

□

The **Cleaning** step cleans the index registers for the Dicke state in a similar fashion as in the **Compress** method in the W-state protocol. In the cleaning process, we have to take the added ordering of the index registers into account. Suppose the l-th qubit of the system register is a 1. If this is the first 1 in the system register, it belongs to index register j_1, and if it is the m-th 1 it belongs to index register j_m. Computing the Hamming weight of the first $l-1$ qubits gives precisely this information. Combined, this shows that if the l-th qubit is a 1 and the Hamming weight of the first $l-1$ qubits equals m, then the l-th qubit should uncompute the $m+1$-th index register j_{m+1}.

Lemma 4.9. *An* LAQCC *circuit exists that implements* ***Cleaning****:*

$$\frac{1}{\sqrt{\binom{n}{k}}}\sum_{j_1<\cdots<j_k}^{n-1}|j_1\rangle\ldots|j_k\rangle|e_{j_1}\oplus\cdots\oplus e_{j_k}\rangle\to\frac{1}{\sqrt{\binom{n}{k}}}\sum_{j_1<\cdots<j_k}^{n-1}|0\rangle\ldots|0\rangle|e_{j_1}\oplus\cdots\oplus e_{j_k}\rangle\,.$$

This circuit uses $\mathcal{O}(n^2\log(n))$ *qubits.*

Proof. The first step, as described above, is to acquire the Hamming weight from all the substrings of the system register. This requires n copies of the system register as well as a $\log(k)$-qubit register to store the Hamming weight value. The copies follow from the fanout-gate.

$$\begin{aligned}
&\binom{n}{k}^{-1/2}\sum_{j_1<\cdots<j_k}|j_1\rangle\ldots|j_k\rangle|e_{j_1}\oplus\cdots\oplus e_{j_k}\rangle|0\rangle_n^{\otimes n-1}|0\rangle_{\log(n)}^{\otimes n}\xrightarrow{(1)}\\
&\binom{n}{k}^{-1/2}\sum_{j_1<\cdots<j_k}|j_1\rangle\ldots|j_k\rangle|e_{j_1}\oplus\cdots\oplus e_{j_k}\rangle^{\otimes n}|0\rangle_{\log(n)}^{\otimes n}\xrightarrow{(2)}\\
&\binom{n}{k}^{-1/2}\sum_{j_1<\cdots<j_k}|j_1\rangle\ldots|j_k\rangle|e_{j_1}\oplus\cdots\oplus e_{j_k}\rangle^{\otimes n}\bigotimes_{l=0}^{n-1}\left||(e_{j_1}\oplus\cdots\oplus e_{j_k})_{[l,n]}|\right\rangle,
\end{aligned}$$

where $|(e_{j_1}\oplus\cdots\oplus e_{j_k})_{[l,n]}|$ denotes the Hamming weight of the substring consisting of qubits l up until n of the system register. Step (1) copies the system qubits using fan-out gates; Step (2) computes the Hamming weight of all the qubits 1 up until $j-1$ using the Hammingweight-gate shown in Table 5; Step (3) cleans the copies of the system register by applying fan-out. This step is omitted from the equations, but is included in the graphical explanation of the circuit, shown in Figure 8 for $n=4$. Note that at the end of the calculation, it is convenient to teleport the Hamming weight registers next to the system register. There are now n new registers containing the information of the Hamming weight, we will refer to them as the Hamming weight registers. This step requires $\mathcal{O}(n^2\log(n))$ qubits.

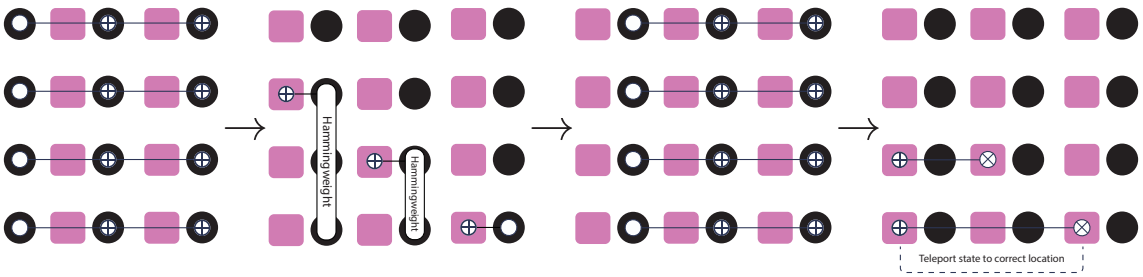


Figure 8: Circuit to implement of the Hamming weight calculation of all qubit strings l to $n-1$ in four steps in parallel. The black dots represent system qubits, the pink squares represent $\log(k)$ qubit registers that can count the Hamming weight up until k.

The last step that remains is to clean the k index registers. Cleaning the k index registers follows similar steps as the **Compress** method in the W-state protocol, with the added Hamming-weight information taken into account. This step requires k copies of the system registers well as k copies of the Hamming-weight registers. Every index register is paired with one copy of the system register and a copy of the n Hamming-weight registers. Cleaning the j-th register consists of five steps, similar to the **Compress** method of the W-state: Step (1) applies Hadamard gates to bring the index register to phase space, in which $CNOT$-gates are diagonalized; Step (2) copies the index register; Step (3) uses the information in the Hamming-weight and system register to apply the phases to the correct index register qubits; Step (4) cleans the index register copies; and, Step (5) applies Hadamard gates to reset the index register qubits to the $|0\rangle$ state

Figure 9 shows the steps taken to clean a single index register j. The black dots represent the qubits in the system register and the upper row of blue dots represent the qubits in index register j. The pink squares represent the ancilla Hamming weight register, where each square represents a

group of $\log(k)$ qubits. This step requires $\mathcal{O}(nk\log(k)\log(n))$ qubits. At the end of the **Cleaning** operation the state is as desired:

$$\frac{1}{\sqrt{\binom{n}{k}}}\sum_{j_1<\cdots<j_k}^{n-1}|0\rangle\ldots|0\rangle\,|e_{j_1}\oplus\cdots\oplus e_{j_k}\rangle\,.$$

The **Cleaning** step requires $\mathcal{O}(n^2\log(n))$ qubits. □

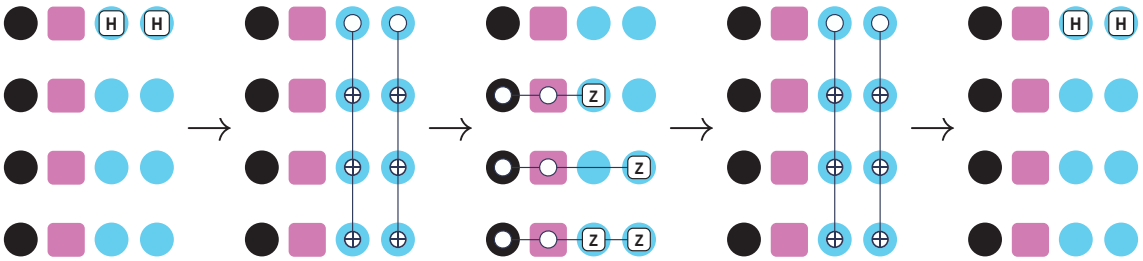


Figure 9: Circuit to clean index register j. The black dots represent qubits in the system register and the blue dots the index register and its copies. The pink squares represent the ancilla Hamming weight register and its copies. Each pink square represents a group of $\log(k)$ qubits. Each of the five grids represents a single timeslice in the circuit.

Theorem 4.10. *For any n and $k=\mathcal{O}(\sqrt{n})$ there exists an* LAQCC *circuit preparing the Dicke-(n,k) state, $|D_k^n\rangle$, using $\mathcal{O}(n^2\log(n))$ qubits.*

Proof. The circuit combines the circuits resulting from Lemmas 4.6, 4.7, 4.8 and 4.9. It consists of four steps:

$$\begin{aligned}
|0\rangle_{i_1}\ldots|0\rangle_{i_k}|0\rangle_{s_1} &\xrightarrow{(1)} \frac{1}{n^{k/2}}\sum_{j_1,\ldots,j_k=0}^{n-1}|j_1\rangle_{i_1}\ldots|j_k\rangle_{i_k}|e_{j_1}\oplus\cdots\oplus e_{j_k}\rangle_{s_1}\\
&\xrightarrow{(2)} \sqrt{\frac{(n-k)!}{n!}}\sum_{j_1\neq\ldots\neq j_k}^{n-1}|j_1\rangle\ldots|j_k\rangle\,|e_{j_1}\oplus\cdots\oplus e_{j_k}\rangle\\
&\xrightarrow{(3)} \frac{1}{\sqrt{\binom{n}{k}}}\sum_{j_1<\cdots<j_k}^{n-1}|j_1\rangle\ldots|j_k\rangle\,|e_{j_1}\oplus\cdots\oplus e_{j_k}\rangle\\
&\xrightarrow{(4)} \frac{1}{\sqrt{\binom{n}{k}}}\sum_{j_1<\cdots<j_k}^{n-1}|0\rangle\ldots|0\rangle\,|e_{j_1}\oplus\cdots\oplus e_{j_k}\rangle
\end{aligned}$$

Step (1) implements **Filling** using Lemma 4.6 requiring $\mathcal{O}(kn\log(n))$ qubits; Step (2) implements **Filtering** using Lemma 4.7 requiring $\mathcal{O}(kn\log(n))$ qubits; Step (3) implements **Ordering** using Lemma 4.8 requiring $\mathcal{O}(k^2\log(n)^2)$ qubits; Step (4) implements **Cleaning** using Lemma 4.9 requiring $\mathcal{O}(n^2\log(n))$ qubits. After every step ancilla qubits are cleaned, so that they can be reused. As $k=\mathcal{O}(\sqrt{n})$ the largest amount of qubits required for a step is Step (4) requiring $\mathcal{O}(n^2\log(n))$ qubits. □

Bärtschi and Eidenbenz posed a conjecture on the optimal depth of quantum circuits that prepare the Dicke-(n,k) state. They give an algorithm for generating Dicke-(n,k) states in depth $\mathcal{O}(k\log(\frac{n}{k}))$, given all-to-all connectivity, and conjecture that this scaling is optimal when k is constant. Our result shows that there is a LAQCC implementation in this regime, when one has access to intermediate measurements and feed forward. This does not disprove their conjecture. However the circuits shown here are also accessible in QNC^1 by Lemma 3.19, giving "pure" quantum circuits with depth $\mathcal{O}(\log(n))$ for $k=\mathcal{O}(\sqrt{n})$ and achieving better scaling when $k=\omega(1)$.

4.4 Dicke states for all k using log-depth quantum circuits

The previous section gave a constant-depth protocol to prepare the Dicke-(n,k) state for $k = \mathcal{O}(\sqrt{n})$. We developed a different method for creating Dicke-(n,k) states which requires logarithmic (in n) quantum depth to prepare Dicke-(n,k), but works for arbitrary k. We first define what we mean with logarithmic quantum depth:

Notation 4.11. We let LAQCC-LOG refer to the instance $\mathsf{LAQCC}(\mathsf{QNC}^0, \mathsf{NC}^1, \mathcal{O}(\log(n)))$, similar to LAQCC except that we allow for a logarithmic number of alterations between quantum and classical calculations. This results in a circuit of logarithmic quantum depth.

In this section we show a LAQCC-LOG circuit that creates the Dicke-(n,k) state.

One way of studying the creation of Dicke states is by looking at efficient algorithms that convert numbers from one representation to another. An example of this is the **Uncompress-Compress** method in the W-state protocol, that converts numbers from a binary representation to a one-hot representation. Dicke states are a generalization of the W-state, hence the one-hot representation no longer suffices for preparing the state. Instead, we use a construction based on number conversion between the combinatorial representation and the factoradic representation. Below we introduce both representations and present quantum circuits that map between the two. Theorem 4.21 proves that a LAQCC-LOG circuit can prepare the Dicke-(n,k) state for any k.

4.4.1 Combinatorial number system

An interesting result showed that any integer $m \geq 0$ can be written as a sum of k binomial coefficients [Bec64]. For fixed k, this is even unique as the next lemma shows.

Lemma 4.12 ([Bec64])**.** *For all integers $m \geq 0$ and $k \geq 1$, there exists a unique decreasing sequence of integers $c_k, c_{k-1}, \ldots, c_1$ with $c_j > c_{j-1}$ and $c_1 \geq 0$ such that*

$$m = \binom{c_k}{k} + \binom{c_{k-1}}{k-1} \cdots \binom{c_1}{1} = \sum_{i=1}^{k} \binom{c_i}{i}.$$

This lemma allows for the definition of the combinatorial number representation:

Definition 4.13. Let $k \in \mathbb{N}$ be a constant. Any integer $m \in \mathbb{N}$ can be represent by a unique string of numbers $(c_k, c_{k-1} \ldots, c_1)$, such that $c_k > c_{k-1} \cdots > c_1 \geq 0$ and $c_k \leq m$. This string is given by the unique decreasing sequence of Lemma 4.12. We call this string the *index representation* denoted by $m^{indx(k)}$.

The bit string of k ones at indices $(c_k, \ldots, c_1)$ is the m-th bit string with k ones in the lexicographical order. This bit string is called the *combinatatorial representation*. We denote the m-th bit string with k ones as $m^{comb(k)}$.

The W-state protocol used the conversion between the binary representation of a number m and its combinatorial representation $m^{comb(1)}$. A generalized number conversion is precisely the protocol needed to prepare Dicke states.

A sketch of the protocol would be as follows: given positive integers k and n: Create a superposition state

$$\binom{n}{k}^{-\frac{1}{2}} \sum_{i=0}^{\binom{n}{k}-1} |i\rangle\,|0\rangle\,;$$

Use number conversion to go from label i to $i^{comb(k)}$

$$\binom{n}{k}^{-\frac{1}{2}} \sum_{i=0}^{\binom{n}{k}-1} |i\rangle \left|i^{comb(k)}\right\rangle;$$

Use number conversion from $i^{comb(k)}$ to i to clean up the label register

$$\binom{n}{k}^{-\frac{1}{2}} \sum_{i=0}^{\binom{n}{k}-1} |0\rangle \left|i^{comb(k)}\right\rangle = |D_k^n\rangle\,.$$

The conversion map from the combinatorial representation to the binary representation is given by Lemma 4.12. This calculation requires iterative multiplication and addition, both of which are in TC^0, hence this calculation is in TC^0.

The converse mapping, from binary to combinatorial representation for given k, can be achieved by a greedy iterative algorithm: On input m, find the biggest c_k such that $m \geq \binom{c_k}{k}$ and subtract this from m: $\tilde{m} = m - \binom{c_k}{k}$. This gives c_k and a residual $\tilde{m}$. Repeat this process for $\tilde{m}$: Find the largest c_j such that $\tilde{m} \geq \binom{c_j}{j}$ and update residual $\tilde{m} = \tilde{m} - \binom{c_j}{j}$, until all c_j are found.

This greedy algorithm is inherently linear in k as it requires all previously found $\{c_i\}_{i=j}^{k}$ to find c_{j-1}. Hence, it is not immediately obvious if and how to achieve this mapping in constant or even logarithmic depth.

4.4.2 Mapping between factoradic representation and combinatorial number system

A number representation closely related to the combinatorial number representation is the *factoradic representation*. This number system uses factorials instead of binomials to represent numbers.

Definition 4.14. A sequence $y = (y_{n-1}, y_{n-2}, \ldots, y_0)$ of integers, such that $j \geq y_j \geq 0$ is called a *factoradic*, or more explicitly an *n-factoradic*. The elements of an n-factoradic is called an *n-digit*. An n-factoradic y can represent a number m between 0 and $n! - 1$, in the following way

$$m = \sum_{j}^{n} y_j \cdot j!. \tag{6}$$

For a given $m \in \{0, \ldots, n!-1\}$, we call the n-factoradic y obeying the equality above, the *factoradic representation of m*. Denote $\mathrm{Fact}(n)$ as the set of all n-factoradics.

The following lemma shows that Equation 6 is a bijection, showing that the factoradic representation is unique.

Lemma 4.15. *For $k \geq 0$ it holds that:*

$$\sum_{i=0}^{k} i \cdot i! = (k+1)! - 1.$$

Proof. Proof by induction.
BASE STEP: Let k be 0:

$$0 \cdot 0! = 1! - 1$$

INDUCTION STEP: Assume the lemma holds for some j, then

$$\sum_{i=0}^{j+1} i \cdot i! = (j+1) \cdot (j+1)! + \sum_{i=0}^{j} i \cdot i = (j+1) \cdot (j+1)! + (j+1)! - 1 = (j+2)! - 1,$$

which completes the proof. □

This identity allows for using factorials as a base for a number system. The next lemma gives a log-space algorithm to convert a factoradic representation to its combinatorial representation.

Lemma 4.16. *There is a logspace algorithm $\mathcal{A}$ that, given $k \in \{0, \ldots, n\}$, and a uniformly random n-factoradic, outputs a uniformly random n-bit string of Hamming weight k.*

Proof. The algorithm $\mathcal{A}$ is given k and an n-factoradic $y = (y_{n-1}, \ldots, y_0)$. It will then output an n-bit string $y^{comb(k)} = y_{n-1}^{comb(k)} \ldots y_0^{comb(k)} \in \{0,1\}^n$ of Hamming weight k, one bit at a time, from left to right, according to the following rule. Let $H_{>n-j} = \sum_{i=n-j+1}^{n-1} y_i^{comb(k)}$ be the Hamming weight of the bits produced before we reach bit $n-j$. Then $y_{n-j}^{comb(k)}$ is given by:

$$(\mathcal{A}(y))_{n-j} = y_{n-j}^{comb(k)} = \begin{cases} 1 & \text{if } y_{n-j} < k - H_{>n-j} \\ 0 & \text{otherwise} \end{cases}. \tag{7}$$

This conversion requires comparing an n-digit with a constant and the Hamming weight of a bitstring. The only information that $\mathcal{A}$ needs to remember, as it goes from bit $n-j+1$ to bit $n-j$, is the Hamming weight $H_{>n-j}$ of the bits it produced so far, and this can be stored in logarithmic space.

Now note that the number of factoradic n-digit strings that map to the same combinatorial bit string is always $k!(n-k)!$: Let $y^{comb(k)} \in \{0,1\}^n$ have Hamming weight k. For any bit position $y_{n-j}^{comb(k)}$, there are $n-j+1-(k-H_{>n-j})$ possible choices for the n-digit $y_{n-j} \in \{0,\ldots,n-j\}$ that set $y_{n-j}^{comb(k)} = 0$. For the leftmost index $n-j$ such that $y_{n-j}^{comb(k)} = 0$, it holds that $H_{>n-j} = j-1$, and then there are $n-k$ possible n-digits y_{n-j} that set $y_{n-j}^{comb(k)} = 0$. Then, for the second index $n-j$ such that $y_{n-j}^{comb(k)} = 0$ it holds that $H_{>n-j} = j-2$, hence there are $n-k-1$ possible n-digits y_{n-j} causing $y_{n-j}^{comb(k)} = 0$. And so forth. This results in $(n-k)!$ different possible choices for the $(n-k)$-many n-digits where $y^{comb(k)} = 0$.

Similarly, for the leftmost position $n-j$ where $y_{n-j}^{comb(k)} = 1$, there are k possible choices for the n-digit y_{n-j} that cause $y_{n-j}^{comb(k)} = 1$. The second leftmost position $n-j$ gives $k-1$ possible choices, and so forth, for a total of $k!$ possible settings of the k-many n-digits where $y^{comb(k)} = 0$.

Combined, we conclude that, for every n-bit string $y^{comb(k)} \in \{0,1\}^n$ of Hamming weight k, there are exactly (the same number of) $k!(n-k)!$ n-factoradics y such that $\mathcal{A}(y) = y^{comb(k)}$. Hence, a uniformly random n-factoradic is mapped by $\mathcal{A}$ to a uniformly random n-bit string of Hamming weight k, as claimed. □

This lemma gives a logspace algorithm to convert a uniformly random n-factoradic to a uniformly random n-bit string of Hamming weight k, for any k. It is well known that logspace is contained in TC^1, allowing this calculation to be performed in parallel log-depth when one has access to threshold gates [Joh90]. As we saw in Section 3.3, we can compute a threshold gate in LAQCC. Hence, an LAQCC-LOG can perform any TC^1 calculation. We conclude:

Corollary 4.17. *The following map can be implemented in* LAQCC-LOG.

$$\frac{1}{\sqrt{n!}} \sum_{y \in Fact(n)} |y\rangle\,|0\rangle \to \frac{1}{\sqrt{n!}} \sum_{y \in Fact(n)} |y\rangle\,|\mathcal{A}(y)\rangle\,.$$

In the next lemma, we show that a TC^0 circuit can implement the inverse of $\mathcal{A}$.

Lemma 4.18. *There exists a TC^0 algorithm which, when given an n-bit string $y^{comb(k)}$ of Hamming weight k, a uniformly-random k-factoradic, and a uniformly-random $(n-k)$-factoradic, outputs a uniformly random n-factoradic y among those such that $\mathcal{A}(y) = y^{comb(k)}$.*

Proof. The conversion can be done in parallel, generating an n-digit for every bit in $y^{comb(k)} = y_{n-1}\ldots y_0 \in \{0,1\}^n$. Recall that we are given as input a uniformly-random k-factoradic $O_{k-1},\ldots,O_0$ and a uniformly-random $(n-k)$-factoradic $Z_{n-k-1},\ldots,Z_0$.

For every bit position $n-j$, for $1 \le j \le n$, calculate the Hamming weight of the bits from $n-j+1$ to $n-1$: $H_{>n-j} = \sum_{i=j+1}^{n-1} y_i^{comb(k)}$. Recall that iterated addition is in TC^0 [Vol99].

If $y_{n-j}^{comb(k)} = 1$, set $y_{n-j} = O_{k-H_{>n-j}}$. This gives a uniform random n-digit between 0 and $k - H_{>n-j} - 1$. If $y_{n-j}^{comb(k)} = 0$, set $y_{n-j} = k - H_{>n-j} + Z_{n-k-H_{>n-j}}$. Note that this gives a uniform random n-digit between $k - H_{>n-j}$ and $n-j$. By construction, it now follows that $\mathcal{A}(y) = y^{comb(k)}$. Computing each n-digit in this way requires summation and indexing, both of which are in $\mathsf{AC}^0 \subseteq \mathsf{TC}^0$ [Vol99]. □

Remark 4.19. *The above algorithm establishes a bijection $(y^{comb(k)}, Z, O) \leftrightarrow y$ between triples $(y^{comb(k)}, Z, O)$ with $y^{comb(k)} \in \{0,1\}^n$ of Hamming weight k, $Z \in Fact(n-k)$ and $O \in Fact(k)$ and n-factoradics $y \in Fact(n)$. Let $(\mathcal{A}(y), \mathcal{Z}(y), \mathcal{O}(y))$ be the image of an n-factoradic y under this bijection. The previous lemma shows that one can compute y from $(y^{comb(k)}, Z, O)$ in TC^0. It is not hard to see that the map $(\mathcal{A}(y), y) \mapsto (\mathcal{A}(y), y, \mathcal{Z}(y), \mathcal{O}(y))$ is also in TC^0. Indeed, to find $\mathcal{Z}(y)$ and $\mathcal{O}(y)$, we need only invert the two defining equalities $y_{n-j} = O_{k-H_{>n-j}}$ and $y_{n-j} = k - H_{>n-j} + Z_{n-k-H_{>n-j}}$.*

Corollary 4.20. *The following map can be implemented in* LAQCC*.*

$$\binom{n}{k}^{-\frac{1}{2}} \sum_{y^{comb(k)}} |0\rangle \left|y^{comb(k)}\right\rangle \to \frac{1}{\sqrt{n!}} \sum_{y \in Fact(n)} |y\rangle |\mathcal{A}(y)\rangle$$

where $y^{comb(k)}$ *ranges over all* n*-bit strings of Hamming weight* k*.*

Proof. The transformation consists of three steps:

$$\begin{aligned}
&\binom{n}{k}^{-\frac{1}{2}} \sum_{y^{comb(k)}} \left|y^{comb(k)}\right\rangle |0\rangle |0\rangle |0\rangle \\
\xrightarrow{(1)}\ &\binom{n}{k}^{-\frac{1}{2}} \sum_{y^{comb(k)}} \left|y^{comb(k)}\right\rangle \left(\bigotimes_{j=0}^{n-k-1} \sum_{i=0}^{j} |i\rangle\right) \left(\bigotimes_{j=0}^{k-1} \sum_{i=0}^{j} |i\rangle\right) |0\rangle \\
=\ &\frac{1}{\sqrt{n!}} \sum_{y^{comb(k)}} \left|y^{comb(k)}\right\rangle \left(\sum_{Z \in \mathrm{Fact}(n-k)} |Z\rangle\right) \left(\sum_{O \in \mathrm{Fact}(k)} |O\rangle\right) |0\rangle \\
\xrightarrow{(2)}\ &\frac{1}{\sqrt{n!}} \sum_{y \in \mathrm{Fact}(n)} |\mathcal{A}(y)\rangle \left|\hat{Z}(y)\right\rangle \left|\hat{O}(y)\right\rangle |y\rangle \\
\xrightarrow{(3)}\ &\frac{1}{\sqrt{n!}} \sum_{y \in \mathrm{Fact}(n)} |\mathcal{A}(y)\rangle |0\rangle |0\rangle |y\rangle
\end{aligned}$$

Step (1) prepares a uniform superposition over all n-factoradics using Theorem 4.1. Step (2) is Lemma 4.18, and Step (3) follows from Remark 4.19. In the above steps we implicitly used that the inverse of the used LAQCC operations are also LAQCC operations. Even though it is unclear if this inverse-property holds in general, it does hold for the considered LAQCC operations. The measurement steps, which might not be reversible, in this algorithm are used to implement fan-out gates. The inverse of a fan-out gate is the fan-out gate itself and hence is contained in LAQCC. □

Theorem 4.21. *There exists a* LAQCC-LOG*-circuit for preparing Dicke-*(n, k) *states, for any positive integers* n *and* $k \leq n$*, it uses* $\mathcal{O}(poly(n))$ *qubits.*

Proof. The circuit combines the circuits resulting from Lemma 4.16 and Lemma 4.18.

It consists of three steps:

$$\begin{aligned}
|0\rangle^{\otimes n \log(n)} |0\rangle^{\otimes n} &\xrightarrow{(1)} \frac{1}{\sqrt{n!}} \left(\bigotimes_{j=0}^{n-1} \sum_{i=0}^{j} |i\rangle\right) |0\rangle^{\otimes n} = \sum_{y \in \mathrm{Fact}(n)} |y\rangle |0\rangle \\
&\xrightarrow{(2)} \frac{1}{\sqrt{n!}} \sum_{y \in \mathrm{Fact}(n)} |y\rangle |\mathcal{A}(y)\rangle \\
&\xrightarrow{(3)} \binom{n}{k}^{-\frac{1}{2}} \sum_{y \in \mathrm{Fact}(n)} |0\rangle |\mathcal{A}(y)\rangle = |D_k^n\rangle .
\end{aligned}$$

Step (1) prepares a uniform superposition over all n-factoradics using Theorem 4.1; Step (2) is by Corollary 4.17; and, Step (3) reverses the algorithm of Corollary 4.20. □

4.5 Quantum many-body scar states

There is a particular set of states in many-body physics called, many-body scar states, which are highly excited states that exhibit atypically low entanglement [Tur+18]. These states exhibit long coherence times relative to other states at the same energy density and seem to avoid thermalization and thereby they do not follow the eigenstate thermalization hypothesis. This makes studying the lifetime of quantum many body scar states under perturbations particularly interesting. Studying

this lifetime is quite challenging, as even though scarred eigenstates often have modest entanglement and therefore have efficient matrix product state representations, perturbations typically couple them to states nearby in energy which typically have volume-law scaling entanglement, making classical simulations difficult.

A overview paper by Gustafson et al. studied methods of preparing quantum many-body scar states on quantum computers, with the goal to simulate time dynamics directly on the quantum system [Gus+23]. They found several approaches for generating quantum many-body scars for a particular model, which require polynomial depth. They look at quantum many-body scar states of the n-qubit spin-1/2 Hamiltonian of [IS20]:

$$H = \lambda \sum_{i=2}^{n-1} (X_i - Z_{i-1} X_i Z_{i+1}) + \Delta \sum_{i=1}^{n} Z_i + J \sum_{i=1}^{n-1} Z_i Z_{i+1}$$

The quantum many-body scar states of n-qubits are given by:

$$|S_k\rangle = \frac{1}{k!\sqrt{\mathcal{N}(n,k)}} (Q^\dagger)^k |\Omega\rangle ,$$

where $\mathcal{N}(n,k) = \binom{n-k-1}{k}$, $|\Omega\rangle = |0\rangle^{\otimes n}$ and $k = 0, \ldots, n/2 - 1$. The raising operator $Q^\dagger$ is given by:

$$Q^\dagger = \sum_{i=2}^{n-1} (-1)^i P_{i-1} \sigma_i^+ P_{i+1},$$

with $P_j = |0\rangle\langle 0|$ and $\sigma_j^+ = X_j + Y_j$. They show that up to local Z gates these states are very closely related to Dicke states:

$$\prod_{\text{i odd}} Z_i |S_k\rangle = |0\rangle \otimes P_{fib} |D_k^n\rangle \otimes |0\rangle ,$$

where P_{fib} is known as the Fibonacci constraint, which is a projector that removes all states where there are two ones next to each other:

$$P_{fib} = I - \sum_{i=1}^{n-1} |11\rangle\langle 11|_{i,i+1} .$$

The goal of this section is to show that these states, for $k = \mathcal{O}(\sqrt{n})$ are accessible in LAQCC. First note that by Theorem 4.10 there exists a LAQCC protocol to generate $|D_k^n\rangle$ up to $k = \mathcal{O}(\sqrt{n})$. We will show that there is a LAQCC protocol that applies P_{fib} to these $|D_k^n\rangle$ states. The first step will be to show that there exists a unitary accessible in LAQCC that flags the correct state.

Lemma 4.22. *There exists a unitary U_{fib}, accessible in* LAQCC*, that flags all the states that obey the Fibonacci constraint, more precisely:*

$$U_{fib} |D_k^n\rangle |0\rangle = \alpha P_{fib} |D_k^n\rangle |0\rangle + \beta (I - P_{fib}) |D_k^n\rangle |1\rangle$$

Proof. Add n extra qubits prepared in $|0\rangle$, one for every sequential pair of qubits. For all $i \in \{1, \ldots, n-1\}$, apply a Toffoli gate with control qubits i and $i+1$ and target qubits the i-th auxillary qubit. The second step is to apply the OR_n gate on the n auxillary qubits. The n-th auxillary qubit is also used as the output qubit. Clean the extra qubits by again applying Toffoli gates. These steps are accessible in LAQCC therefore implements the flag unitary with an LAQCC protocol. □

The second step is to show that α is bounded by a constant in the case that $k = \mathcal{O}(\sqrt{n})$, this is shown using the following two lemma's.

Lemma 4.23. *The total number of bitstrings of length n with k ones, such that no two ones are adjacent is given by:*

$$\binom{n-k}{k} + \binom{n-k-1}{k-1}$$

Proof. We can count the number of possible bitstrings, after first noticing that every 1 must be followed by a 0, unless the last bit is a 1. As a result, we have two situations, in the first, we can consider all possible rearrangements $n-k$ elements, consisting of k pairs '10' and $n-2k$ ones. This gives $\binom{n-k}{k}$ possible bitstrings. In the second situation, the last bit is 1. This leave $k-1$ pairs '10' in a total of $n-k-1$ elements. With the same reasoning, this gives $\binom{n-k-1}{k-1}$ possible bitstrings. Summing the two situations proves the lemma. □

We now consider the relative fraction of this type of bitstrings among all possible bitstrings with Hamming-weight k.

Lemma 4.24. *Let $k = c\sqrt{n}$ for some constant $c > 0$. Then the following inequality holds*

$$\frac{\binom{n-k}{k} + \binom{n-k-1}{k-1}}{\binom{n}{k}} \geq \exp\big(-c^2\big)$$

Proof. We have

$$\frac{\binom{n-k}{k} + \binom{n-k-1}{k-1}}{\binom{n}{k}} > \frac{\binom{n-k}{k}}{\binom{n}{k}} = \frac{(n-k)!/k!(n-2k)!)}{n!/k!(n-k!)} = \frac{(n-k)!^2}{n!(n-2k)!}.$$

Expanding the factorials and only consider the terms that do not cancel, we obtain

$$\frac{(n-k)!}{n!}\frac{(n-k)!}{(n-2k)!} = \frac{(n-k)(n-k-1)\dots(n-(2k-1))}{n(n-1)\dots(n-(k-1))}.$$

Both the numerator and denominator have K terms, which we can pair. next we note that $\frac{a}{b} > \frac{a-1}{b-1}$ whenever $b > a$ (and $b \notin \{0,1\}$). Using this idea, we obtain the following expression:

$$\frac{n-k}{n}\frac{n-k-1}{n-1}\dots\frac{n-(2k-1)}{n-(k-1)} > (\frac{n-k}{n})^k = (1-\frac{k}{n})^k.$$

Now using that $k = c\sqrt{n}$, we have

$$\left(1-\frac{c\sqrt{n}}{n}\right)^{c\sqrt{n}} = \left(\left(1-\frac{c}{\sqrt{n}}\right)^{\frac{\sqrt{n}}{c}}\right)^{c^2} > \exp\big(-c^2\big),$$

which is a constant. □

This allows us construct the state $|S_k\rangle$ using the steps for the Dicke-state preparation together with Lemma 3.14. Note that Lemma 3.14 requires us to implement both U and $U^\dagger$, where U implements the initial superposition. The **Ordering**-step (Lemma 4.8) does however use measurements, which stops us makes implementing the inverse of the circuit hard. Still, we can work around this, by applying Lemma 3.14 between the **Filtering** and **Ordering** step:

Theorem 4.25. *For any n and $k = \mathcal{O}(\sqrt{n})$ there exists a* LAQCC *circuit preparing the many-body scar state $|S_k\rangle$, using $\mathcal{O}(n^2\log(n))$ qubits.*

Proof. We follow the same steps as for the Dicke-state preparation (see Theorem 4.10). After the second **Filtering** step however, we apply use the unitary U_{fib} together with Lemma 3.14 to filter out all states with subsequent ones in the state. Note that by Lemma 4.24, the number of states with no consecutive ones is a constant fraction of the total number of strings of Hamming-weight k.

Furthermore, the state after the Filtering step,

$$\sqrt{\frac{(n-k)!}{(n)!}}\sum_{j_1\neq\dots\neq j_k}|j_1\rangle\dots|j_k\rangle\,|e_{j_1}\oplus\dots\oplus e_{j_k}\rangle\,,$$

is still entangled with the index registers. In effect there are many copies of the Dicke-(n,k) state in on the system register, each with a diferent ordering of the index registers, however this does not affect the fraction of states with no consecutive ones compared to the states with consecutive ones. Next, the **Ordering** and **Cleaning** step of the protocol work similarly on the resulting state and will give the state $|S_k\rangle$. □

Acknowledgements

We want to thank Jonas Helsen, Joris Kattemölle, Ido Niesen, Kareljan Schoutens, Florian Speelman, Dyon van Vreumingen and Jordi Weggemans for insight full discussions. Furthermore, we would like to thank Georgios Styliaris for first mentioning how to parallelize Clifford ladder circuits. HB and MF were supported by the Dutch Ministry of Economic Affairs and Climate Policy (EZK), as part of the Quantum Delta NL programme. BL was funded by the European Union (ERC, HOFGA, 101041696). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them. Also supported by FCT through the LASIGE Research Unit, ref. UIDB/00408/2020 and ref. UIDP/00408/2020. NN was supported by the quantum application project of TNO. This work was supported by the Dutch Research Council (NWO/OCW), as part of the Quantum Software Consortium programme (project number 024.003.037).

References

[Aar04] S. Aaronson. "Multilinear Formulas and Skepticism of Quantum Computing". In: *Proceedings of the Thirty-Sixth Annual ACM Symposium on Theory of Computing*. STOC '04. Chicago, IL, USA: Association for Computing Machinery, 2004, pp. 118–127. DOI: 10.1145/1007352.1007378.

[AAS20] S. Aaronson, Y. Atia, and L. Susskind. *On the Hardness of Detecting Macroscopic Superpositions*. 2020. DOI: 10.48550/arXiv.2009.07450. arXiv: 2009.07450 [quant-ph].

[Aff+87] I. Affleck et al. "Rigorous results on valence-bond ground states in antiferromagnets". In: *Phys. Rev. Lett.* 59 (7 Aug. 1987), pp. 799–802. DOI: 10.1103/PhysRevLett.59.799.

[AK07] S. Aaronson and G. Kuperberg. "Quantum versus Classical Proofs and Advice". In: *Twenty-Second Annual IEEE Conference on Computational Complexity (CCC'07)*. 2007, pp. 115–128. DOI: 10.1109/CCC.2007.27.

[BCH06] D. Bacon, I. L. Chuang, and A. W. Harrow. "Efficient Quantum Circuits for Schur and Clebsch-Gordan Transforms". In: *Phys. Rev. Lett.* 97 (17 Oct. 2006), p. 170502. DOI: 10.1103/PhysRevLett.97.170502.

[BE19] A. Bärtschi and S. Eidenbenz. "Deterministic preparation of Dicke states". In: *International Symposium on Fundamentals of Computation Theory*. Springer. 2019, pp. 126–139. DOI: 10.1007/978-3-030-25027-0˙9.

[BE22] A. Bärtschi and S. Eidenbenz. *Short-Depth Circuits for Dicke State Preparation*. 2022. DOI: 10.1109/QCE53715.2022.00027.

[Bec64] E. F. Beckenbach. *Applied combinatorial mathematics*. New York, J. Wiley, 1964, pp. 27–30.

[Bez+21] A. V. Bezvershenko et al. "Dicke Transition in Open Many-Body Systems Determined by Fluctuation Effects". In: *Phys. Rev. Lett.* 127 (17 Oct. 2021), p. 173606. DOI: 10.1103/PhysRevLett.127.173606.

[BJS10] M. J. Bremner, R. Jozsa, and D. J. Shepherd. "Classical simulation of commuting quantum computations implies collapse of the polynomial hierarchy". In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 467.2126 (Aug. 2010), pp. 459–472. DOI: 10.1098/rspa.2010.0301.

[BKP11] D. Browne, E. Kashefi, and S. Perdrix. "Computational Depth Complexity of Measurement-Based Quantum Computation". In: *Theory of Quantum Computation, Communication, and Cryptography*. Ed. by W. van Dam, V. M. Kendon, and S. Severini. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 35–46. DOI: 10.1007/978-3-642-18073-6˙4.

[BMS17] M. J. Bremner, A. Montanaro, and D. J. Shepherd. "Achieving quantum supremacy with sparse and noisy commuting quantum computations". In: *Quantum* 1 (Apr. 2017), p. 8. DOI: 10.22331/q-2017-04-25-8.

[Bra+22] S. Brandhofer et al. “Benchmarking the performance of portfolio optimization with QAOA”. In: *Quantum Information Processing* 22.1 (Dec. 2022). DOI: 10.1007/s11128-022-03766-5.

[Bra13] T. Brandes. “Excited-state quantum phase transitions in Dicke superradiance models”. In: *Phys. Rev. E* 88 (3 Sept. 2013), p. 032133. DOI: 10.1103/PhysRevE.88.032133.

[CEB20] J. Cook, S. Eidenbenz, and A. Bärtschi. “The Quantum Alternating Operator Ansatz on Maximum k-Vertex Cover”. In: *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. 2020, pp. 83–92. DOI: 10.1109/QCE49297.2020.00021.

[CJL08] S. Clark, R. Jozsa, and N. Linden. “Generalized Clifford groups and simulation of associated quantum circuits”. In: *Quantum Inf. Comput.* 8.1 (2008), pp. 106–126. DOI: 10.26421/QIC8.1-2-8.

[Dic54] R. H. Dicke. “Coherence in Spontaneous Radiation Processes”. In: *Phys. Rev.* 93 (1 Jan. 1954), pp. 99–110. DOI: 10.1103/PhysRev.93.99.

[DVC00] W. Dür, G. Vidal, and J. I. Cirac. “Three qubits can be entangled in two inequivalent ways”. In: *Phys. Rev. A* 62 (6 Nov. 2000), p. 062314. DOI: 10.1103/PhysRevA.62.062314.

[Gar+23] G. García-Pérez et al. *Experimentally accessible nonseparability criteria for multipartite-entanglement-structure detection*. Mar. 2023. DOI: 10.1103/PhysRevResearch.5.013226.

[GC99] D. Gottesman and I. L. Chuang. “Demonstrating the viability of universal quantum computation using teleportation and single-qubit operations”. In: *Nature* 402.6760 (1999), pp. 390–393. DOI: 10.1038/46503.

[Got98] D. Gottesman. “The Heisenberg Representation of Quantum Computers”. In: *arXiv e-prints*, quant-ph/9807006 (July 1998), quant–ph/9807006. DOI: 10.48550/arXiv.quant-ph/9807006. arXiv: `quant-ph/9807006 [quant-ph]`.

[Gre+02] F. Green et al. “Counting, fanout and the complexity of quantum ACC”. In: *Quantum Inf. Comput.* 2.1 (2002), pp. 35–65. DOI: 10.26421/QIC2.1-3.

[Gro96] L. K. Grover. “A Fast Quantum Mechanical Algorithm for Database Search”. In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*. STOC ’96. Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, pp. 212–219. DOI: 10.1145/237814.237866.

[Gus+23] E. J. Gustafson et al. “Preparing quantum many-body scar states on quantum computers”. In: *Quantum* 7 (Nov. 2023), p. 1171. DOI: 10.22331/q-2023-11-07-1171.

[Had+19] S. Hadfield et al. “From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz”. In: *Algorithms* 12.2 (Feb. 2019), p. 34. DOI: 10.3390/a12020034.

[Häf+05] H. Häffner et al. “Scalable multiparticle entanglement of trapped ions”. In: *Nature* 438.7068 (Dec. 2005), pp. 643–646. DOI: 10.1038/nature04279.

[HŠ05] P. Høyer and R. Špalek. “Quantum Fan-out is Powerful”. In: *Theory of Computing* 1.5 (2005), pp. 81–103. DOI: 10.4086/toc.2005.v001a005.

[IS20] T. Iadecola and M. Schecter. “Quantum many-body scar states with emergent kinetic constraints and finite-entanglement revivals”. In: *Phys. Rev. B* 101 (2 Jan. 2020), p. 024306. DOI: 10.1103/PhysRevB.101.024306.

[Joh90] D. S. Johnson. “A Catalog of Complexity Classes”. In: *Algorithms and Complexity*. Elsevier, 1990, pp. 67–161. DOI: 10.1016/b978-0-444-88071-0.50007-2.

[Joz06] R. Jozsa. “An introduction to measurement based quantum computation”. In: *NATO Science Series, III: Computer and Systems Sciences. Quantum Information Processing-From Theory to Experiment* 199 (2006), pp. 137–158. DOI: 10.48550/arXiv.quant-ph/0508124.

[Kit97] A. Y. Kitaev. “Quantum computations: algorithms and error correction”. In: *Russian Mathematical Surveys* 52.6 (Dec. 1997), pp. 1191–1249. DOI: 10.1070/rm1997v052n06abeh002155.

[Lon01] G. L. Long. “Grover algorithm with zero theoretical failure rate”. In: *Phys. Rev. A* 64 (2 July 2001), p. 022307. DOI: 10.1103/PhysRevA.64.022307.

[Moo99] C. Moore. “Quantum Circuits: Fanout, Parity, and Counting”. In: *arXiv preprint arXiv:quant-ph/9903046* (1999). DOI: 10.48550/arXiv.quant-ph/9903046. eprint: arXiv:quant-ph/9903046.

[Mou+18] S. Moudgalya et al. “Exact excited states of nonintegrable models”. In: *Phys. Rev. B* 98 (23 Dec. 2018), p. 235155. DOI: 10.1103/PhysRevB.98.235155.

[Mou+20] S. Moudgalya et al. “Large classes of quantum scarred Hamiltonians from matrix product states”. In: *Phys. Rev. B* 102 (8 Aug. 2020), p. 085120. DOI: 10.1103/PhysRevB.102.085120.

[MR18] D. Maslov and M. Roetteler. “Shorter Stabilizer Circuits via Bruhat Decomposition and Quantum Circuit Transformations”. In: *IEEE Transactions on Information Theory* 64.7 (2018), pp. 4729–4738. DOI: 10.1109/TIT.2018.2825602.

[MRB18] S. Moudgalya, N. Regnault, and B. A. Bernevig. “Entanglement of exact excited states of Affleck-Kennedy-Lieb-Tasaki models: Exact results, many-body scars, and violation of the strong eigenstate thermalization hypothesis”. In: *Phys. Rev. B* 98 (23 Dec. 2018), p. 235156. DOI: 10.1103/PhysRevB.98.235156.

[MY23] T. Metger and H. Yuen. “stateQIP = statePSPACE”. In: (2023), pp. 1349–1356. DOI: 10.1109/FOCS57990.2023.00082.

[NC10] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010. DOI: 10.1017/CBO9780511976667.

[Nee+10] M. Neeley et al. “Generation of three-qubit entangled states using superconducting phase qubits”. In: *Nature* 467.7315 (Sept. 2010), pp. 570–573. DOI: 10.1038/nature09418.

[NM14] Y. Nakata and M. Murao. “Diagonal quantum circuits: Their computational power and applications”. In: *The European Physical Journal Plus* 129.7 (July 2014). DOI: 10.1140/epjp/i2014-14152-9.

[ÖSI07] S. K. Özdemir, J. Shimamura, and N. Imoto. “A necessary and sufficient condition to play games in quantum mechanical settings”. In: *New Journal of Physics* 9.2 (Feb. 2007), pp. 43–43. DOI: 10.1088/1367-2630/9/2/043.

[Ouy14] Y. Ouyang. “Permutation-invariant quantum codes”. In: *Phys. Rev. A* 90 (6 Dec. 2014), p. 062317. DOI: 10.1103/PhysRevA.90.062317.

[PB10] M. Plesch and V. Bužek. “Efficient compression of quantum information”. In: *Phys. Rev. A* 81 (3 Mar. 2010), p. 032317. DOI: 10.1103/PhysRevA.81.032317.

[Pre+09] R. Prevedel et al. “Experimental Realization of Dicke States of up to Six Qubits for Multiparty Quantum Networking”. In: *Phys. Rev. Lett.* 103 (2 July 2009), p. 020503. DOI: 10.1103/PhysRevLett.103.020503.

[PS13] P. Pham and K. M. Svore. “A 2D nearest-neighbor quantum architecture for factoring in polylogarithmic depth”. In: *Quantum Inf. Comput.* 13.11-12 (2013), pp. 937–962. DOI: 10.26421/QIC13.11-12-3.

[PSC21] L. Piroli, G. Styliaris, and J. I. Cirac. “Quantum Circuits Assisted by Local Operations and Classical Communication: Transformations and Phases of Matter”. In: *Phys. Rev. Lett.* 127 (22 Nov. 2021), p. 220503. DOI: 10.1103/PhysRevLett.127.220503.

[RB01] R. Raussendorf and H. J. Briegel. “A One-Way Quantum Computer”. In: *Phys. Rev. Lett.* 86 (22 May 2001), pp. 5188–5191. DOI: 10.1103/PhysRevLett.86.5188.

[RY22] G. Rosenthal and H. S. Yuen. “Interactive Proofs for Synthesizing Quantum States and Unitaries”. In: *13th Innovations in Theoretical Computer Science Conference (ITCS 2022)*. Ed. by M. Braverman. Vol. 215. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, 112:1–112:4. DOI: 10.4230/LIPIcs.ITCS.2022.112.

[SB09] D. J. Shepherd and M. J. Bremner. "Temporally unstructured quantum computation". In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 465 (2009), pp. 1413–1439. DOI: 10.1098/rspa.2008.0443.

[Sho97] P. W. Shor. "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer". In: *SIAM J. Comput.* 26.5 (1997), pp. 1484–1509. DOI: 10.1137/S0097539795293172.

[SI19] M. Schecter and T. Iadecola. "Weak Ergodicity Breaking and Quantum Many-Body Scars in Spin-1 *XY* Magnets". In: *Phys. Rev. Lett.* 123 (14 Oct. 2019), p. 147201. DOI: 10.1103/PhysRevLett.123.147201.

[Smi+23] K. C. Smith et al. "Deterministic Constant-Depth Preparation of the AKLT State on a Quantum Processor Using Fusion Measurements". In: *PRX Quantum* 4 (2 Apr. 2023), p. 020315. DOI: 10.1103/PRXQuantum.4.020315.

[Sus18] L. Susskind. *Three Lectures on Complexity and Black Holes*. 2018. DOI: 10.48550/arXiv.1810.11563. arXiv: 1810.11563 [hep-th].

[Tan+24] N. Tantivasadakarn et al. "Long-Range Entanglement from Measuring Symmetry-Protected Topological Phases". In: *Phys. Rev. X* 14 (2 June 2024), p. 021040. DOI: 10.1103/PhysRevX.14.021040.

[TDL10] O. Tsyplyatyev, J. von Delft, and D. Loss. "Simplified derivation of the Bethe-ansatz equations for the Dicke model". In: *Phys. Rev. B* 82 (9 Sept. 2010), p. 092203. DOI: 10.1103/PhysRevB.82.092203.

[Tót12] G. Tóth. "Multipartite entanglement and high-precision metrology". In: *Phys. Rev. A* 85 (2 Feb. 2012), p. 022322. DOI: 10.1103/PhysRevA.85.022322.

[TT13] Y. Takahashi and S. Tani. "Collapse of the Hierarchy of Constant-Depth Exact Quantum Circuits". In: *2013 IEEE Conference on Computational Complexity*. 2013, pp. 168–178. DOI: 10.1109/CCC.2013.25.

[Tur+18] C. J. Turner et al. "Weak ergodicity breaking from quantum many-body scars". In: *Nature Physics* 14.7 (2018), pp. 745–749. DOI: 10.1038/s41567-018-0137-5.

[TVV23a] N. Tantivasadakarn, R. Verresen, and A. Vishwanath. *Shortest Route to Non-Abelian Topological Order on a Quantum Processor*. Aug. 2023. DOI: 10.1103/PhysRevLett.131.060405.

[TVV23b] N. Tantivasadakarn, A. Vishwanath, and R. Verresen. "Hierarchy of Topological Order From Finite-Depth Unitaries, Measurement, and Feedforward". In: *PRX Quantum* 4.2 (June 2023). DOI: 10.1103/prxquantum.4.020339.

[Van+21] J. S. Van Dyke et al. "Preparing Bethe Ansatz Eigenstates on a Quantum Computer". In: *PRX Quantum* 2 (4 Nov. 2021), p. 040329. DOI: 10.1103/PRXQuantum.2.040329.

[Vol99] H. Vollmer. *Introduction to circuit complexity: a uniform approach*. Springer, 1999. DOI: 10.1007/978-3-662-03927-4.

[Wat+19] A. B. Watts et al. "Exponential separation between shallow quantum circuits and unbounded fan-in shallow classical circuits". In: *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*. ACM, June 2019, pp. 515–526. DOI: 10.1145/3313276.3316404.

A Useful lemmas

This section gives two lemmas. The first lemma upper bounds the computational power of LAQCC. The second lemma helps in preparing Dicke states for $k \in \mathcal{O}(\sqrt{n})$.

Lemma A.1. *Let* $\Pi = (\Pi_{yes}, \Pi_{no})$ *be a decision problem in* NC^1. *Then there is a uniform log-depth quantum circuit that decides on* Π.

Proof. Let B be the uniform Boolean circuit of logarithmic depth deciding on Π. As $\Pi \in \mathsf{NC}^1$, such a circuit exists.

For fixed input size n, write B as a Boolean tree of depth $\mathcal{O}(\log(n))$, with at its leaves the n input bits x_i and as root an output bit. This Boolean tree directly translates in a classical circuit using layers of AND, OR and NOT gates.

Each of these gates has a direct quantum equivalent gate, provided that we use ancilla qubits: First replace all OR gates by AND and NOT gates. Then replace all AND gates by Tofolli gates, which has three inputs. The third input is a clean ancilla qubit and will store the AND of the other two inputs. Finally, replace all NOT gates by X-gates. □

Lemma A.2. *Let $n, k \in \mathbb{N}$ and $k < \frac{n}{2}$ then:*

$$\frac{n!}{n^k(n-k)!} > e^{-\frac{2k^2}{n}}$$

Proof. The result follows by a simple computation

$$\begin{aligned}\frac{n!}{n^k(n-k)!} &= e^{\sum_{i=1}^{k} \log(1-\frac{i}{n})} \\ &> e^{\sum_{i=1}^{k} \frac{-i}{n-i}} \\ &> e^{\sum_{i=1}^{k} \frac{-i}{n-k}} \\ &> e^{-\frac{k^2}{n-k}} \\ &> e^{-\frac{2k^2}{n}},\end{aligned}$$

where we use that $\log(1+x) \geq \frac{x}{1+x}$ for $x > -1$.

□

B Gate implementations

B.1 OR-gate

In this section we discuss the implementation of the OR_n-gate and why we can implement it using local gates in a nearest-neighbor architecture. We show how the gate works for any basis state $|x\rangle = |x_1\rangle \otimes \ldots \otimes |x_n\rangle$. By linearity, the gate then works for arbitrary superpositions. The OR-gate by Takahashi and Tani consists of two steps: First they apply the OR-reduction introduced in Ref. [HŠ05], which prepares a state on $\log n$ qubits such that the OR evaluated on these $\log n$ qubits yields the same result as the OR evaluated on the original n qubits. Second, they evaluate an exponential circuit on these $\log n$ qubits to calculate the OR-gate. This results in a polynomial-sized circuit for the OR-gate.

Let $m = \lceil \log_2(n+1) \rceil$, then the OR-reduction implements the map

$$|x\rangle |0\rangle^{\otimes m} \to |x\rangle \bigotimes_{j=1}^{m} \left|\mu_{\phi_j}^{|x|}\right\rangle,$$

where $\varphi_j = \frac{2\pi}{2^j}$ and $\left|\mu_{\varphi}^{|x|}\right\rangle = HR_Z(\varphi \cdot |x|)H|0\rangle$. The OR-reduced state thus depends on the weighted Hamming weight of x, more precisely, every $\left|\mu_{\phi_j}^{|x|}\right\rangle$ depends on the entire string x. For every $\left|\mu_{\phi_j}^{|x|}\right\rangle$ this requires a copy of $|x\rangle$ which can be created by using the fanout gate. The circuit applying the rotation $R_Z(\varphi \cdot |x|)$ consists of sequential R_Z gates, which are diagonal and hence can be implemented in parallel by Lemma 2.8. This results in a circuit of width $\mathcal{O}(n \log(n))$ for creating the OR-reduced state.

We have for every bitstring $x \in \{0,1\}^n$ that

$$\mathrm{OR}_n(x) = \frac{1}{2^{n-1}} \sum_{a \in \{0,1\}^n \setminus \{0^n\}} \mathrm{PA}_n^a(x),$$

where $\mathrm{PA}_n^a(x) = \oplus_{j=0}^{n-1} a_i x_i$ is the parity of x, weighted by the non-zero binary vector a. Hence, computing the OR of the input is now reduced to computing all parities of the subsets of the inputs. The parity gate is equivalent to a fanout-gate conjugated by Hadamard gates on every qubit, see also Table 2.

We now copy the m qubits of the OR-reduced state $2^m = n$ times, using fanout gates. For each copy we require two additional auxiliary qubits. The first will hold the result of the parity computation, for which we already have a nearest-neighbor implementation. We will entangle the second auxiliary qubit using a fanout-gate to obtain a GHZ-state in these qubits.

We now compute in parallel the parity of the subsets of the inputs. For every subset we use a single copy of the inputs and we store the result in the corresponding auxiliary qubit. We then apply a controlled-R_Z gate from the first auxiliary qubit to the second auxiliary qubit in the GHZ state. This prepared the state (omitting other registers)

$$\frac{1}{\sqrt{2}}(|0\rangle^{n-1} + (-1)^{\frac{1}{2^{n-1}}\sum_{a\in\{0,1\}^m\setminus\{0^m\}}\mathrm{PA}_m^a(x)}|1\rangle^{\otimes n-1}) = \frac{1}{\sqrt{2}}(|0\rangle^{n-1} + (-1)^{\mathrm{OR}_n(x)}|1\rangle^{\otimes n-1}).$$

Uncomputing this final state using a fanout-gate gives a single qubit that holds the desired answer in its phase. A single Hadamard gate applied to this qubit will then give the answer in a single qubit.

Combining all steps thus gives an implementation for the OR gate using a geometrically local nearest-neighbor circuit. For more details on the implementation as well as a proof of correctness, we refer to the original proof [TT13].

B.2 Equality-gate

Define the Equality gate on two n-qubit computational basis states as

$$\text{Equality} : |x\rangle|y\rangle|0\rangle \mapsto |x\rangle|y\rangle|\mathbb{1}_{x=y}\rangle\,.$$

This gate is implemented in three steps: (1) subtract the first register from the second using a subtraction circuit; (2) apply Equal_0 on the second register and store the result in the third register; (3) add the first register to the second, undoing the subtraction computation:

$$\begin{aligned}|x\rangle|y\rangle|0\rangle &\xrightarrow[(1)]{} |x\rangle|y-x\rangle|0\rangle\\ &\xrightarrow[(2)]{} |x\rangle|y-x\rangle|\mathbb{1}_{x=y}\rangle\\ &\xrightarrow[(3)]{} |x\rangle|y\rangle|\mathbb{1}_{x=y}\rangle\end{aligned}$$

Addition and subtraction both have width $\mathcal{O}(n^2)$, which, as a result, the Equality-gate also has.

B.3 Greaterthan-gate

Define the Greaterthan gate on two n-qubit computational basis states as

$$\text{Greaterthan} : |x\rangle|y\rangle|0\rangle \mapsto |x\rangle|y\rangle|\mathbb{1}_{x>y}\rangle\,.$$

This gate is implemented in four step: (1) Add an extra clean qubit to the second register and interpret this as an $n+1$-qubit register with most significant bit zero; (2) subtract the first register from the second. The subtraction is modulo 2^{n+1}; (3) apply a CNOT-gate from most significant bit of the second register to the third register; (4) add the first register to the second, undoing the subtraction computation:

$$\begin{aligned}|x\rangle|y\rangle|0\rangle &\xrightarrow[(1)]{} |x\rangle|0y\rangle|0\rangle\\ &\xrightarrow[(2)]{} |x\rangle\left|y-x \bmod 2^{n+1}\right\rangle|0\rangle\\ &\xrightarrow[(2)]{} |x\rangle\left|y-x \bmod 2^{n+1}\right\rangle|\mathbb{1}_{x>y}\rangle\\ &\xrightarrow[(3)]{} |x\rangle|0\rangle|y\rangle|\mathbb{1}_{x>y}\rangle\end{aligned}$$

This construction works, as after step (2), the most significant bit of the second register is one, precisely if x is larger than y.

Addition and subtraction both have width $\mathcal{O}(n^2)$, which, as a result, the Greaterthan-gate also has.

B.4 Exact$_t$-gate

Define the Exact$_t$ gate on an n-qubit computational basis state as

$$\text{Exact}_t : |x\rangle |0\rangle \mapsto |x\rangle \left|\mathbb{1}_{|x|=t}\right\rangle .$$

Here, $|x|$ denotes the Hamming weight of the n-bit string x.

This gate follows by combining the Hammingweight-gate and the Equality-gate: First, compute the Hamming weight of x in an ancilla register and then apply the Equality gate to check that this ancilla register equals t.

Another approach is to modify the circuit for OR slightly. In the OR-reduction step, add a gate $R_Z(-\varphi t)$, which adjusts the angle to be zero precisely if $|x| = t$ (see Theorem 4.6 of [HŠ05]). Then apply the circuit for OR and negate the output. The circuit for OR evaluates to zero, precisely if the input had Hamming weight t.

B.5 Threshold$_t$-gate

Define the Threshold$_t$ gate on an n-qubit computational basis state as

$$\text{Threshold}_t : |x\rangle |0\rangle \mapsto |x\rangle \left|\mathbb{1}_{|x|\geq t}\right\rangle .$$

Taking the OR over the outputs of Exact$_j$-gates for all $j \geq t$, gives the Threshold$_t$-gate. An improved implementation with better scaling in t is given in Theorem 2 of [TT13].

A weighted threshold gate uses weights w_i and evaluates to one precisely if $\sum_i w_i x_i \geq t$. Assume without loss of generality that $\sum_i w_i x_i$ evaluates to an integer. Otherwise, we can use the same ideas, but up to some precision.

Use the same OR-reduction as for the normal threshold gate. Instead of rotations $R_Z(\varphi)$ controlled by x_i, we use rotations $R_Z(w_i\varphi)$ controlled by x_i. This implements the weighted threshold gate.